TECHNICAL SPECIFICATION · DOCUMENT 5 OF 6

Wellness Studio OS

API Contract, Integration & Communication Document

REST contracts · ERPNext/Frappe integration · AI and agent APIs · n8n automation · WhatsApp, email and in-app communication · Razorpay · webhooks · security · MVP scope

Item	Detail
Document	Document 5 of 6 — API Contract, Integration & Communication Document
Version / Date	1.0 · 29 September 2026
Sources of truth	Master PRD v1.0 (incl. Appendix A) · Solution Architecture Document v1.0 · Module Specification v1.0 · Database Design Document v1.0. No other source is used.
Scope	Interface contracts only. PRD requirements, architecture decisions and database schemas are referenced by ID and not restated.
Audience	Backend, frontend, ERP/Frappe, AI, automation and QA engineers of the 6-member team.
Priority notation	P0 = mandatory for the 5-Day MVP · P1 = important (Phase 1 additions / first sprint after MVP) · P2 = Phase 2 or later. (Matches Documents 3 and 4; PRD P1 maps to P0/P1 here.)
Status	Implementation-ready draft for CTO / mentor review

Assumptions and source gaps (recorded so the team does not re-litigate)

#	Item	Treatment in this document
C5-01	The sources fix the API version prefix (/api/v1), the Razorpay webhook path (/api/webhooks/razorpay) and internal ERP callback paths (/internal/events/*), but no other URL paths.	All other paths below are the contract proposed by this document. Once approved they are frozen in packages/contracts (OpenAPI) and generated into the typed client.
C5-02	Individual permission codes are only exemplified in the sources (members:create, bookings:on_behalf, payments:record, attendance:correct). The catalogue is code-seeded as resource:action.	Codes used here follow the same pattern; the seeded iam.permissions catalogue must be reconciled with §3–§4 before Day 1 freeze.
C5-03	Email provider, phone-OTP provider and LLM provider are not selected (Architecture S7, PRD Appendix C).	Contracts are provider-agnostic; provider adapters sit behind integrations/email, integrations/llm and the OTP adapter.
C5-04	Exact ERP DocType-to-field mapping is a Day-1 task (Architecture §7.1). The ERP methods named in Module Spec §9.2 are used verbatim; their payloads are defined here at contract level.	ERP method payloads are stable at the adapter boundary; internal DocType shape may change without changing public APIs.
C5-05	Module Spec lists Notification Log as ERP DocType; Document 4 (D-01) resolved it to the App DB.	This document follows Document 4: notification records live in notify.* and are exposed only via the API.
C5-06	Email and in-app notification centre are Phase 2 (PRD Part B/D). Session-summary and agent APIs are Phase 2+.	Specified now for contract stability; flagged P1/P2 in §14 and disabled by feature flag.
C5-07	Razorpay refunds/subscriptions are Phase 2; Phase 1 recurring debit is out of scope (PRD).	Contracts defined (§11) but marked P2; 'Subscriptions' covers SaaS tier billing and opt-in recurring debit, both P2.

Table of Contents

Assumptions and source gaps (recorded so the team does not re-litigate) . . . 1

Table of Contents **2**

1. API Standards **6**

1.1 API design principles . . . 6

1.2 REST conventions . . . 6

1.3 Naming standards . . . 6

1.4 Versioning strategy . . . 7

1.5 Response format . . . 7

1.6 Error format . . . 7

1.7 Pagination, filtering and sorting . . . 8

2. Authentication & Authorization **9**

2.1 JWT strategy . . . 9

2.2 RBAC model . . . 9

2.3 Tenant security . . . 10

2.4 Authentication and role-management endpoints . . . 10

3. B2B APIs (Studio Management Portal) **13**

3.1 Dashboard (B2B-01) . . . 13

3.2 CRM and Leads (B2B-07) . . . 13

3.3 Members (B2B-02) . . . 14

3.4 Memberships (B2B-03) . . . 15

3.5 Classes (B2B-04) . . . 15

3.6 Bookings (B2B-05) . . . 16

3.7 Attendance (B2B-06) . . . 17

3.8 Events (B2B-08) . . . 17

3.9 Payments (B2B-09) . . . 18

3.10 Reports (B2B-10) . . . 19

3.11 Staff (B2B-11) . . . 19

3.12 Notifications (B2B-13) . . . 20
3.13 Settings — Configuration Center (B2B-12) . . . 20
3.14 Tenant Management (B2B-14 — platform-operator side) . . . 21
4. B2C APIs (Member Portal and Public Site) 22
4.1 Dashboard (B2C-01) . . . 22
4.2 Memberships (B2C-02) . . . 22
4.3 Bookings (B2C-03) . . . 22
4.4 Attendance (B2C-04) . . . 23
4.5 Events (B2C-05) . . . 23
4.6 Payments (B2C-06) . . . 23
4.7 Profile (B2C-07) . . . 24
4.8 Notifications (B2C-08) . . . 24
4.9 Recommendations (B2C-09) . . . 25
4.10 Public (Visitor) APIs — no login, tenant from host . . . 25
5. ERP Integrations (ERPNext / Frappe) 26
5.1 Adapter transport contract . . . 26
5.2 Integration catalogue . . . 26
5.3 Frappe (custom app) responsibilities and ERP → API callback contract . . . 28
5.4 ERPNext vs Frappe — what each integration touches . . . 28
6. AI Service APIs 29
6.1 Common AI contract . . . 29
6.2 AI Assistant (admin) — B2B-15 . . . 29
6.3 AI Enquiry Assistant (public) . . . 30
6.4 AI Booking Assistant (member) . . . 30
6.5 AI Recommendation Assistant (member) . . . 31
6.6 AI Feedback Assistant . . . 31
7. Future Agent APIs (Phase 2+) 32
7.1 Common agent API contract . . . 32

7.2 Agent catalogue . . . 32

8. Automation & n8n Integrations **34**

8.1 Event envelope and dispatcher contract . . . 34

8.2 Internal automation API (n8n → NestJS) . . . 34

8.3 Workflow contracts . . . 35

8.4 Event catalogue . . . 36

9. WhatsApp Communication Architecture **37**

9.1 Architecture and shared rules . . . 37

9.2 Workflows by domain . . . 37

10. Email & In-App Notifications **40**

10.1 Supporting endpoints (email and in-app) . . . 40

11. Payment Integration (Razorpay) **42**

11.1 Payment state machine . . . 42

11.2 Contracts . . . 42

12. Webhook Specifications **44**

12.1 Razorpay (inbound) . . . 44

12.2 ERP events (inbound callbacks) . . . 44

12.3 Notification events (WhatsApp status webhook, inbound) . . . 44

12.4 Outbound event webhooks (booking, membership, attendance, notification) . . . 44

13. Error Handling & Security **46**

13.1 Error code catalogue . . . 46

13.8 Security controls . . . 47

14. MVP API Scope **49**

14.1 Core APIs . . . 49

14.2 ERP APIs . . . 49

14.3 AI APIs . . . 49

14.4 Agent APIs . . . 50

14.5 Automation APIs . . . 50

14.6 Communication APIs 50
14.7 Delivery mapping (PRD 5-day plan) and ownership 50
14.8 QA and integration acceptance checklist 51

1. API Standards

1.1 API design principles

- **One front door (AD03).** Browsers, n8n, AI and agents reach business capabilities only through the NestJS API. The ERP is never exposed; the adapter uses a restricted integration identity.
- **Same capability, every channel.** REST controllers and AI tools call the same domain services (Architecture §3.2). An AI tool has no capability without a REST-equivalent service path, and inherits identical validation, permissions and audit.
- **Contract-first.** The OpenAPI 3.1 spec in packages/contracts is authored before implementation; the typed client and DTO validators are generated from it. Breaking changes require a new version.
- **Thin API layer.** Controllers validate, authenticate, bind tenant and call services. No business decision in controllers (BR-BKG-01: the ERP transaction is the final validator).
- **Server-generated identifiers (BR-GEN-04).** Clients never send an ID on creation. ERP-owned records use naming-series IDs (e.g. MEM-00042); App-DB records use UUIDs.
- **Tenant from host, never from body (AD11).** The tenant is resolved from the sub-domain and bound to the JWT; a tenant identifier in a request body or query is ignored and rejected if it conflicts.
- **Commit first, notify after (AD06).** Write endpoints return the authoritative result; events are emitted to the outbox after commit. Side-effect failures never change the API result.
- **Fail visibly, never falsely.** Unknown state is returned as pending/failed with an explicit code; an ERP outage returns 503 ERP_UNAVAILABLE and never a cached confirmation (NFR-AVL-03).
- **Idempotent by design (NFR-REL-01).** All non-safe endpoints accept an Idempotency-Key; webhooks are de-duplicated on provider event ID.

1.2 REST conventions

Topic	Convention
Base URLs	Staff/member API: https://{tenant-slug}.{platform-domain}/api/v1 · Provider webhooks: /api/webhooks/{provider} (unversioned, signature-secured) · Internal callbacks (ERP/n8n → API): /internal/events/{type} and /internal/automation/* (service credentials only, not reachable from the browser).
Route groups	/api/v1/auth/* · /api/v1/b2b/* (staff portal) · /api/v1/b2c/* (member portal) · /api/v1/public/* (visitor; no login; tenant from host) · /api/v1/ai/* · /api/v1/agents/* · /api/v1/platform/* (Platform Administrator).
Methods	GET read (safe, cacheable only for config) · POST create/action · PUT full replace (rare) · PATCH partial update · DELETE archive (soft) — hard delete is not exposed (BR-GEN-03).
Actions	State transitions and commands are POST sub-resources with a verb: /memberships/{id}/approve, /bookings/{id}/cancel, /attendance/{id}/correct.
Content type	application/json; charset=utf-8 only (multipart only for logo/photo upload: POST /b2b/settings/branding/assets). Max request body 1 MB (5 MB for assets).
Time and money	Timestamps ISO-8601 UTC (2026-10-05T12:30:00Z); business dates as YYYY-MM-DD interpreted in the branch time zone (default Asia/Kolkata). Money is a decimal string with currency ("amount":"1500.00","currency":"INR"); Razorpay calls convert to integer paise inside the adapter.
Phone / email	Phone in E.164 (+919876543210); e-mail lower-cased and trimmed server-side.
Idempotency	Header Idempotency-Key (UUID) on every POST/PATCH. Same key + same body within 24 h returns the stored response (Idempotent-Replayed: true); same key + different body returns 409 IDEMPOTENCY_KEY_REUSED. Stored in ops.idempotency_keys.
Correlation	Header X-Correlation-ID accepted/generated on every request, propagated to ERP, outbox, n8n and WhatsApp callbacks, and echoed in meta.correlation_id (NFR-AUD-04).
Concurrency	Updatable resources return an ETag; PATCH requires If-Match. Mismatch → 412 PRECONDITION_FAILED.
Caching	Authenticated responses are Cache-Control: no-store. Booking availability and eligibility are never cached (Architecture §7.3).

1.3 Naming standards

Element	Standard	Example
URL path	lowercase kebab-case, plural nouns, max 3 nesting levels	/b2b/class-sessions/{id}/roster

Element	Standard	Example
JSON fields	snake_case; booleans prefixed is_/has_; timestamps suffixed _at; dates suffixed _date	starts_at, is_active, expiry_date
Enumerations	lower_snake_case strings, documented per resource (never numeric codes)	pending_approval, active, expired
IDs	field id (own) and <entity>_id (reference); ERP references are strings	member_id: "MEM-00042"
Query params	snake_case; filter[field]=value; sort=-field,field; q= free text; page, page_size; cursor for streams	?filter[status]=active&sort=-created_at
Events	<aggregate>.<past_tense_verb>, lower-case dotted	booking.confirmed, membership.expiring
Error codes	UPPER_SNAKE_CASE, stable, prefixed by domain	BOOKING_CLASS_FULL
Permissions	<resource>:<action>	members:create, bookings:on_behalf
Headers	Standard HTTP names; custom headers X- prefixed	X-Correlation-ID, X-Signature

1.4 Versioning strategy

- **URI major version** (/api/v1). A new major version is created only for breaking changes; v(N-1) is supported for at least 6 months after v(N) GA and announces retirement with Deprecation and Sunset headers.
- **Non-breaking changes** (new optional request fields, new response fields, new enum values on outputs, new endpoints) ship within v1. Clients must ignore unknown response fields. New enum values are announced in the OpenAPI changelog.
- **Breaking** = removing/renaming fields or endpoints, tightening validation, changing types, error codes or semantics.
- **ERP contract version** is separate: the adapter checks the custom app version at start-up (Architecture §11) and records it in ops.erp_contract_versions; deploy order is ERP custom app → migrations → API/worker → web → n8n (Architecture §13.3).
- **Webhook/event schema version** is carried in the envelope field schema_version (integer); consumers must handle N and N-1.
- **Feature flags** (platform.tenant_feature_flags) gate unfinished or tier-restricted endpoints; a gated endpoint returns 403 FEATURE_NOT_ENTITLED.

1.5 Response format

All JSON responses use one envelope. Lists carry pagination in meta; the data field is never null on success (empty list = []).

```
// 200 OK — single resource
{ "success": true,
  "data": { "id": "MEM-00042", "full_name": "Rahul Menon", "status": "active", "created_at": "2026-10-05T09:12:44Z" },
  "meta": { "correlation_id": "c1f0a5e2-…", "request_id": "req_8fd2…", "server_time": "2026-10-05T09:12:44Z" } }

// 200 OK — collection
{ "success": true,
  "data": [ { "id": "MEM-00042", "…": "…" } ],
  "meta": { "correlation_id": "…", "pagination": { "page": 1, "page_size": 25, "total_items": 132, "total_pages": 6, "has_next": true } } }

// 201 Created — Location header /api/v1/b2b/members/MEM-00043 ; 202 Accepted — async job (data.job_id, poll status URL)
// 204 No Content — used only for logout and DELETE/archive
```

Status codes: 200, 201, 202, 204; 400 malformed; 401 unauthenticated; 403 not permitted; 404 not found (also for cross-tenant); 409 conflict/duplicate/state; 412 ETag mismatch; 413 too large; 415 media type; 422 validation; 423 locked; 429 rate limited; 500 internal; 502 upstream (ERP/provider) error; 503 ERP or dependency unavailable; 504 upstream timeout.

1.6 Error format

```
{ "success": false,
  "error": {
    "code": "VALIDATION_FAILED",           // stable, machine-readable (catalogue §13)
```

```
  "message": "One or more fields are invalid.",   // safe, human-readable, no internals
  "details": [ { "field": "email", "code": "INVALID_FORMAT", "message": "Enter a valid email address." } ],
  "retryable": false,                    // true for 429/502/503/504 class
  "retry_after_seconds": null,
  "correlation_id": "c1f0a5e2-…" } }
```

Rules: field-level details for 422 (B2B-MEM-01); ERP rule failures are mapped to stable application codes with the ERP reason preserved in details[].reason (e.g. full, ineligible, duplicate); stack traces, SQL and ERP tracebacks are never returned; AI tools relay these messages verbatim (B2B-AIA-02).

1.7 Pagination, filtering and sorting

Feature	Contract
Offset pagination (default for admin lists)	page (default 1), page_size (default 25, max 100). Response meta.pagination as in §1.5. Requests beyond the last page return an empty list.
Cursor pagination (streams: audit log, delivery log, message log)	cursor (opaque), limit (default 50, max 200). Response meta.next_cursor; null when exhausted. Stable under inserts.
Filtering	filter[field]=value, filter[field][in]=a,b, filter[date][gte]=2026-10-01, filter[date][lte]=…; unknown filter fields → 422 UNKNOWN_FILTER. Each list endpoint below documents its allowed filters.
Search	q= matches indexed text fields per resource (e.g. members: name, email, phone, ID). Minimum 2 characters.
Sorting	sort=field or sort=-field (descending); multiple comma-separated; only whitelisted fields; default sort is documented per list (usually -created_at).
Field selection	fields=id,full_name,status (optional; permission filtering still applies).
Scope filtering	Tenant/studio/branch and role scope are applied server-side before filters; a Trainer’s lists are automatically restricted to assigned sessions (BR-ATT-05).

2. Authentication & Authorization

2.1 JWT strategy

Aspect	Specification
Tokens	Short-lived access JWT (15 min, RS256/ES256, key rotation via kid) + opaque rotating refresh token (staff 12 h idle / 7 d absolute; members 30 d absolute). Refresh tokens are stored only as hashes in iam.auth_sessions (UNIQUE refresh_token_hash).
Delivery	HTTP-only, Secure, SameSite=Lax cookies (wsos_at, wsos_rt) bound to the tenant host. Bearer header accepted for non-browser clients (n8n uses service credentials, not user JWTs).
Access-token claims	iss, aud (api), sub (user UUID), tid (tenant UUID), typ (staff | member | platform | service), roles[] (role codes), scope {studio_ids[], branch_ids[]}, sid (auth session UUID), erp_ref (MEM-/STF- ID when linked), mfa (bool), jti, iat, exp.
What is NOT in the token	Individual permissions and grants. They are resolved on every request from the policy service (cached ≤ 30 s, invalidated on change), so a role change takes effect on the next request (B2B-STF-01) and grants are revocable instantly.
Tenant binding	The tid claim must equal the tenant resolved from the Host header; mismatch → 404 NOT_FOUND and a security event (audit.security_events).
Refresh rotation	Each refresh issues a new refresh token and revokes the previous. Re-use of a revoked token revokes the entire session family (theft detection) and returns 401 AUTH_REFRESH_REUSED.
Revocation	Logout, password reset, deactivation, role removal or admin 'revoke sessions' set revoked_at; the auth guard checks sid against a short-TTL revocation cache.
Service tokens	n8n, ERP callbacks and agents use iam.service_principals credentials: n8n → API via signed requests (HMAC-SHA256 over body + timestamp) or client-credentials JWT with typ=service and a tenant-scoped, domain-scoped scope list; rotated and non-interactive.
MFA	TOTP for Owner/Admin: enforcement is P2 (mandatory before production per NFR-SEC-02). Endpoints exist from P1 with mfa flag in the token; when enforced, login returns AUTH_MFA_REQUIRED and a challenge.

2.2 RBAC model

- **Roles** (iam.roles): Owner, Admin, Front Desk, Trainer, Member, Visitor (implicit, unauthenticated) and operator-side Platform Administrator. Roles are bundles of permissions (iam.role_permissions).
- **Evaluation chain** on every request (Architecture §12.2): Identity → Role → Permission → Tenant/Studio/Branch scope → Resource ownership. Members act only on records linked to their own erp_ref; Trainers only on assigned sessions.
- **Special grants** (iam.permission_grants): explicit, scoped and revocable, e.g. bookings:on_behalf for a Trainer; each grant/revoke is audited. Front Desk AI read queries (ai:query) and membership approval by Front Desk require an Owner grant (PRD §7.4 footnote).
- **Permission codes** follow resource:action with risk_class read | write | sensitive (used to filter AI tools). Guards call the central policy service; no inline role checks.
- **Non-bypassable rules** (eligibility, payment, capacity) live in the ERP transaction; no permission overrides them (BR-BKG-06).

Capability	Owner	Admin	Front Desk	Trainer	Member	Visitor
members:*	all	all except role admin	create/read/update	read (assigned)	own	—
memberships:*	all	all + approve	create/renew; approve only if granted	—	own view/buy	—
classes:*	all	all	view	assigned	view/book	public view
bookings:*	all	all	on_behalf	only if granted	own	—
attendance:*	all	mark/correct	check-in	mark assigned	own history	—
events:*	all	all	register others	assigned	register	public events
payments:*	all	operational	record/collect	—	own	—
crm/leads:*	all	all	capture/update	—	—	submit enquiry

Capability	Owner	Admin	Front Desk	Trainer	Member	Visitor
reports:*	all	operational	limited	limited (no finance)	—	—
staff/roles/settings	full	limited (no roles/integrations)	—	—	—	—
audit:read	full	limited	—	—	—	—
ai:*	all	all	read queries only if granted	—	—	enquiry only

2.3 Tenant security

- Tenant resolved from Host → request-scoped context → PostgreSQL SET LOCAL app.tenant_id (RLS on every table) → adapter injects tenant into every ERP call (permission_query_conditions on all Wellness DocTypes) → AI retrieval and n8n workflows carry tenant ID.
- Cross-tenant attempts return 404 NOT_FOUND (not 403), are written to audit.security_events, and alert on repetition (BR-GEN-02).
- Platform Administrator endpoints (/api/v1/platform/*) never return tenant business data; support access is time-boxed and audited (PRD §7.3).
- Public routes accept only the host-derived tenant; bot protection and rate limits apply.

2.4 Authentication and role-management endpoints

All endpoints below live under /api/v1. Unless stated, every response uses the envelope of §1.5 and every request carries X-Correlation-ID. Auth endpoints are rate-limited per IP and per identifier (§13.8).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /auth/staff/login`	Staff e-mail + password login	{ email, password }	200 { access_expires_at, user{id,name,roles[],scope}, must_reset_password, mfa_required } + cookies	email format; password 1–128 chars; tenant from host; lockout after 5 failures / 15 min	public	401 AUTH_INVALID_CREDENTIALS · 423 AUTH_ACCOUNT_LOCKED · 403 AUTH_ACCOUNT_DEACTIVATED · 401 AUTH_MFA_REQUIRED · 429	P0
`POST /auth/member/otp/request`	Registration + login step 1: send OTP to phone	{ phone (E.164), purpose: login|register|link }	202 { challenge_id, expires_in: 300, resend_after: 30 }	E.164; max 5 challenges/phone/hour; OTP stored hashed; response identical whether phone exists or not	public	422 VALIDATION_FAILED · 429 OTP_RATE_LIMITED · 502 OTP_PROVIDER_FAILED	P0
`POST /auth/member/otp/verify`	Registration + login step 2: verify OTP, create or link account	{ challenge_id, otp (6 digits), full_name? , email? (register only) }	200 { user, is_new_user, member_id, must_complete_profile } + cookies	otp 6 digits, ≤ 5 attempts, expiry 5 min; linking matches on verified phone and never creates a duplicate member (B2C-PRF-01)	public	401 OTP_INVALID · 410 OTP_EXPIRED · 409 CONFLICT_DUPLICATE · 429	P0
`POST /auth/member/google`	Optional Google sign-in / link	{ id_token }	200 as verify; 202 { challenge_id } when phone verification is required to link	id_token signature, aud and expiry checked with Google; e-mail verified claim required; account linking requires verified phone	public	401 AUTH_INVALID_CREDENTIALS · 409 ACCOUNT_LINK_REQUIRED	P1
`POST /auth/register/staff-invite (accept)`	Staff account activation from Owner invite (staff registration)	{ invite_token, password, full_name }	200 { user } + cookies	password ≥ 12 chars, not in breach list, 3 of 4 classes; token single-use, expires 72 h	public (token)	410 INVITE_EXPIRED · 422 · 409	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /auth/refresh`	Rotate refresh token, issue new access token	cookie wsos_rt (or { refresh_token })	200 { access_expires_at } + rotated cookies	token hash exists, not revoked/expired, session active, tenant matches host	refresh token	401 AUTH_REFRESH_INVALID · 401 AUTH_REFRESH_REUSED · 401 AUTH_TOKEN_EXPIRED	P0
`POST /auth/logout`	Revoke current session (optionally all)	{ all_sessions?: bool }	204	valid session	authenticated	401	P0
`POST /auth/password/forgot`	Start staff password reset	{ email }	202 (always, non-enumerating)	email format; 3 requests/hour/e-mail	public	429 RATE_LIMITED	P0
`POST /auth/password/reset`	Complete reset with token	{ token, new_password }	204; all sessions revoked	token single-use, 30 min expiry; password policy as above; not equal to last 5	public (token)	410 RESET_TOKEN_EXPIRED · 422 PASSWORD_POLICY	P0
`POST /auth/password/change`	Mandatory / voluntary password change	{ current_password, new_password }	204	current password correct; policy; clears must_reset_password	authenticated (staff)	401 · 422 PASSWORD_POLICY	P0
`GET /auth/me`	Current identity, effective permissions, tenant theme flags	—	200 { user, roles[], permissions[], scope, features[], tenant{name,branding_ref} }	—	authenticated	401	P0
`POST /auth/mfa/enroll · /verify`	TOTP enrolment / challenge for privileged staff	{ } / { code }	200 { otpauth_uri } / 200 tokens	6-digit TOTP, 30 s window ±1	Owner, Admin	401 MFA_INVALID	P2

Role and permission management (Owner only unless stated)

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /b2b/roles`	List roles with permission counts	filter[is_system], page	200 [{ id, code, name, is_system, permission_count }]	—	roles:read	403	P0
`POST /b2b/roles`	Create custom role (bundle)	{ code, name, permission_codes[] }	201 role	code unique per tenant, slug pattern; permission codes exist in catalogue; cannot exceed creator's own permissions	roles:manage	409 CONFLICT_DUPLICATE · 422 · 403	P1
`PATCH /b2b/roles/{id}`	Rename / change permission set	{ name?, permission_codes[]? }	200 role	system roles' permission set is immutable except Owner-approved tier features; audited before/after	roles:manage	409 ROLE_IN_USE_IMMUTABLE · 412	P1
`GET /b2b/permissions`	Permission catalogue (resource:action, risk_class)	filter[resource]	200 [{ code, resource, action, risk_class, description }]	—	roles:read	403	P0
`PUT /b2b/users/{id}/roles`	Assign roles with studio/branch scope	{ assignments:[{ role_code, studio_ids[], branch_ids[] }] }	200 { user_id, assignments[] } effective on next request	role exists; scopes belong to tenant; Admin cannot assign Owner/Admin; last Owner cannot be removed	roles:assign	403 · 409 LAST_OWNER · 422	P0
`GET /b2b/users/{id}/grants`	List active special grants	—	200 [grant]	—	roles:read	404	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /b2b/users/{id}/grants`	Create scoped grant (e.g. bookings:on_behalf)	{ permission_code, studio_id?, branch_id?, expires_at?, reason }	201 grant	permission grantable; scope ⊂ user scope; reason required; audited	roles:assign	422 · 403	P0
`DELETE /b2b/users/{id}/grants/{grant_id}`	Revoke grant (row retained with revoked_at)	—	204	grant active	roles:assign	404 · 409	P0
`POST /b2b/users/{id}/sessions/revoke`	Force sign-out of a user	—	204	—	roles:assign	404	P1

3. B2B APIs (Studio Management Portal)

Base path `/api/v1/b2b`. Every endpoint is tenant-scoped (host), permission-checked, audit-logged for writes, returns the §1.5 envelope, and supports loading/empty/error/permission-denied UI states (PRD §9). Common errors on every endpoint: 401 AUTH_TOKEN_EXPIRED, 403 PERMISSION_DENIED, 404 NOT_FOUND, 429 RATE_LIMITED, 503 ERP_UNAVAILABLE (ERP-backed endpoints). Rows list only endpoint-specific errors. Pri = P0/P1/P2 per §14.

3.1 Dashboard (B2B-01)

Served by the Reporting service (same service used by the AI Assistant, guaranteeing identical figures — B2B-DSH-01/RPT-03). ERP aggregates may be cached ≤ 60 s; Trainers receive own-session data only and no revenue.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /dashboard/kpis`	Six KPI tiles for the caller's role and scope	period=today|week|month|custom, from, to, studio_id?, branch_id?	200 { active_members, active_memberships, todays_attendance, monthly_revenue{amount,currency}, upcoming_events, expiring_memberships{count,window_days}, as_of }	period enum; from ≤ to; max range 366 d	dashboard:read (revenue tile also reports:financial)	422 · 403 (tile omitted, not error, when not permitted)	P0
`GET /dashboard/quick-actions`	Role-aware quick actions (Add Member, Add Class, Add Event)	—	200 [{ code, label, route, permission }]	—	dashboard:read	—	P0
`GET /dashboard/today`	Today's sessions and check-in status (Trainer/Front Desk home)	studio_id?, branch_id?	200 { sessions:[{ session_id, class_type, starts_at, trainer, capacity, booked, checked_in }] }	Trainer auto-filtered to assigned	classes:read	—	P0
`GET /dashboard/pipeline`	Renewal pipeline (upcoming/overdue/completed) with drill-down filters	window_days, status	200 { upcoming, overdue, completed, list_href }	window_days 1–90	memberships:read	422	P2

3.2 CRM and Leads (B2B-07)

ERP methods: wellness_studio.api.lead.create_lead (duplicate check) and lead.convert_lead (source preserved). Leads are never Members before conversion (BR-CRM-01/02). lead.created is emitted to the outbox after commit. The public form endpoint is in §4.10.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /leads`	List/search leads	q, filter[status], filter[source], filter[campaign], filter[created_at][gte/lte], page, sort	200 [lead{id (LEAD-NNNNN), name, phone, email, source, campaign, status, owner, created_at, duplicate_of?}]	allowed filters only	leads:read	422 UNKNOWN_FILTER	P0
`POST /leads`	Capture lead (walk-in / phone / manual)	{ name, phone, email?, source, campaign?, interest?, consent{whatsapp:bool}, notes? }	201 lead; 200 with duplicate_flag when merged/flagged	name 2–120; phone E.164; email format; source in configured list; consent required before follow-up (BR-CRM-04); client cannot send id	leads:create	409 LEAD_DUPLICATE (flag/merge policy) · 422	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /leads/{id}	Lead detail with activity history	—	200 lead + activities[] + conversion?	—	leads:read	404	P0
PATCH /leads/{id}	Update status/owner/notes	{ status?, owner_id?, notes? } + If-Match	200 lead	status ∈ new, contacted, trial, qualified, lost, converted (converted only via /convert)	leads:update	409 INVALID_STATE_TRANSITION · 412	P0
POST /leads/{id}/convert	Explicit lead → member conversion (history preserved)	{ member{ …as POST /members without id }, trigger: manual|membership_purchase, plan_id? }	201 { member_id, lead_id, lead_status: converted, source_retained: true }	same rules as member creation (duplicate email/phone); lead must not already be converted	leads:convert + members:create	409 CONFLICT_DUPLICATE · 409 LEAD_ALREADY_CONVERTED · 422	P0
POST /leads/{id}/merge	Merge duplicate leads	{ merge_into_id }	200 lead	both leads same tenant, not converted	leads:update	409 · 404	P1
GET /leads/sources	Source list and conversion by source (attribution)	from, to	200 [{ source, campaign, leads, converted, rate }]	range ≤ 366 d	reports:read	422	P2

3.3 Members (B2B-02)

ERP method: wellness_studio.api.member.create_member (atomic duplicate check on email/phone). ID format MEM-NNNNN generated by ERP. Archive instead of delete (BR-GEN-03); every change audited with before/after.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /members	List, search, filter members	q, filter[status]=active|inactive|frozen, filter[joined_at], filter[plan_id], page, page_size, sort	200 [member{id,full_name,email,phone,status,join_date,active_membership{plan,expiry}}]	trainer: only members booked in assigned sessions	members:read	422	P0
POST /members	Create member (walk-in registration)	{ full_name, email, phone, date_of_birth?, gender?, join_date?, status?, emergency_contact{name,phone}?, health_notes?, photo_asset_id? }	201 member (id MEM-NNNNN)	name, email, phone required; email format; phone E.164; date not future; status enum; no id accepted (BR-GEN-04)	members:create	409 CONFLICT_DUPLICATE (details.field=email|phone, existing_id) · 422 · 403	P0
GET /members/{id}	Member profile	—	200 member (health_notes only with members:read_sensitive)	—	members:read	404	P0
PATCH /members/{id}	Update member	partial fields + If-Match	200 member	as create; duplicate re-checked when email/phone change	members:update	409 · 412 · 422	P0
POST /members/{id}/archive	Archive (soft delete); restore via /restore	{ reason }	200 { status: archived }	reason required; no active future bookings unless force by Owner	members:archive (Owner/Admin)	409 MEMBER_HAS_ACTIVE_BOOKINGS · 403 (Front Desk)	P0
GET /members/{id}/history	Memberships, bookings, attendance, payments, communication tabs	tab, page	200 [items] per tab	tab enum	members:read (+ payments:read for payments tab)	403	P1
POST /members/import	Bulk import (validate every row)	multipart file (CSV), dry_run=true|false	202 { job_id } → GET /jobs/{id} { accepted, rejected[{row, errors}] }	same duplicate rules per row; no partial silent failure	members:import	413 · 415 · 422	P2

3.4 Memberships (B2B-03)

ERP methods: membership.activate_membership, expire_membership, suspend_membership. Payment state and approval state are independent (BR-MSP-02, BR-PAY-03). Plans are ERP business configuration. Events: membership.activated / .expiring / .expired.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /membership-plans`	List plans	filter[is_active]	200 [plan{id,name,price,currency,validity_days,allowed_class_types[],booking_limit,is_active}]	—	memberships:read	—	P0
`POST /membership-plans`	Create plan (no deployment needed)	{ name, price, validity_days, allowed_class_type_ids[], booking_limit?, requires_approval:bool, is_active }	201 plan	price ≥ 0; validity ≥ 1; class types exist; name unique per tenant	settings:plans	409 · 422	P0
`PATCH /membership-plans/{id}`	Edit plan (does not alter existing memberships, BR-MSP-06)	partial + If-Match	200 plan	audited; existing memberships unchanged	settings:plans	412 · 422	P0
`GET /memberships`	List memberships	filter[status], filter[expiring_in_days], filter[member_id], filter[plan_id], q	200 [membership{id,member_id,plan,status,payment_status,approval_status,start_date,end_date}]	expiring_in_days 1–90	memberships:read	422	P0
`POST /memberships`	Assign membership to member	{ member_id, plan_id, start_date, end_date?, payment_mode: online|manual|pending }	201 membership (status pending_approval or active)	member exists; end > start; price ≥ 0; no overlapping active same-plan membership	memberships:create	404 MEMBER_NOT_FOUND · 409 MEMBERSHIP_OVERLAP · 422	P0
`POST /memberships/{id}/approve`	Approve pending membership (activation only if payment satisfied or waived)	{ note? }	200 { status: active } (or pending_approval + payment_status if unpaid)	state must be pending_approval; approver holds memberships:approve	memberships:approve	409 INVALID_STATE_TRANSITION · 403	P0
`POST /memberships/{id}/renew`	Manual renewal (new validity + payment record)	{ plan_id?, payment_mode }	201 { membership, payment_id? }	membership Active or Expired within grace; creates payment record	memberships:renew	409 · 422	P0
`POST /memberships/{id}/suspend · /cancel`	Suspend / cancel with reason	{ reason }	200 membership	reason ≥ 5 chars; state allows	memberships:manage	409 · 422	P1
`GET /memberships/expiring`	Expiring-soon list (dashboard tile, AI query source)	days=7, page	200 [membership + member]	days 1–90	memberships:read	422	P0
`POST /memberships/{id}/freeze · /unfreeze`	Freeze extends validity	{ from, to, reason }	200	—	memberships:manage	—	P2

3.5 Classes (B2B-04)

Class types and schedules are ERP configuration; capacity default 30 comes from config.tenant_settings and is overridable per class/session (BR-BKG-02). Class types are archived, never deleted.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /class-types · POST /class-types · PATCH /class-types/{id}	List/create/edit/archive class types (Dance, Yoga, Zumba seeds; Pilates etc. by config)	{ name, code, default_capacity?, colour?, is_active }	200/201 class_type	name unique; capacity 1–500	settings:class_types	409 · 422 · 409 CLASS_TYPE_IN_USE (archive, not delete)	P0
GET /class-sessions	Schedule with availability	from, to, filter[class_type_id], filter[trainer_id], filter[status], studio_id	200 [session{id,class_type,starts_at,ends_at,trainer,capacity,confirmed,waitlisted,status}]	range ≤ 62 d; Trainer auto-scoped	classes:read	422	P0
POST /class-schedules	Create one-off or recurring schedule (expands to sessions)	{ class_type_id, trainer_id, start_time, duration_min, capacity?, recurrence{ freq: none|weekly, days[], until } , studio_id, branch_id }	201 { schedule_id, sessions_created }	trainer active; until ≤ 6 months; no trainer double-booking	classes:create	409 TRAINER_CONFLICT · 422	P0
PATCH /class-sessions/{id}	Edit session (time, trainer, capacity)	partial + If-Match	200 session	capacity < confirmed bookings blocked (CAPACITY_BELOW_BOOKINGS) unless handling=waitlist_overflow	classes:update	409 CAPACITY_BELOW_BOOKINGS · 412	P0
POST /class-sessions/{id}/cancel	Cancel a session and notify booked members	{ reason }	200 session status cancelled; bookings released	session in future	classes:update	409 · 422	P1
GET /class-sessions/{id}/roster	Roster with booking/attendance state	—	200 { session, participants:[{ member_id, name, booking_status, attended }], waitlist[] }	Trainer: assigned sessions only	classes:read	403 NOT_ASSIGNED	P0
GET /class-sessions/{id}/qr	Generate rotating check-in QR token for the session	—	200 { qr_payload, expires_at }	token valid for session window; HMAC-signed	attendance:qr_generate	409 SESSION_NOT_OPEN	P0

3.6 Bookings (B2B-05)

ERP method: booking.create_booking (row-locks session; eligibility, limit, duplicate, capacity in one transaction — NFR-PRF-03), booking.cancel_booking, booking.promote_waitlist. Events: booking.confirmed, booking.waitlisted, booking.cancelled, waitlist.promoted (emitted after commit). Requires Idempotency-Key.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /bookings	Book on behalf of a member (Admin/Front Desk/granted Trainer)	{ member_id, session_id, join_waitlist_if_full?: bool }	201 { id (BKG-NNNNN), status: confirmed|waitlisted, waitlist_position?, session, membership_id }	ids exist; membership active & entitled; limit; no duplicate; capacity — final decision by ERP at commit	bookings:on_behalf	409 BOOKING_CLASS_FULL · 409 BOOKING_DUPLICATE · 422 BOOKING_NOT_ELIGIBLE (details.reason: no_membership|expired|suspended|plan_excludes_class|limit_reached) · 403	P0
GET /bookings	List bookings	filter[member_id], filter[session_id], filter[status], from, to	200 [booking]	—	bookings:read	422	P0
GET /bookings/{id}	Booking detail	—	200 booking	—	bookings:read	404	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /bookings/{id}/cancel`	Cancel booking (frees seat, promotes next eligible)	{ reason? }	200 { status: cancelled, promoted_booking_id? }	cancel window: cutoff default 240 min before start, configurable (BR-BKG-03); Owner/Admin may override with reason	bookings:cancel	409 BOOKING_CUTOFF_PASSED · 409 INVALID_STATE_TRANSITION	P0
`GET /class-sessions/{id}/eligibility`	Eligibility check for a member/session (read-only; used by UI and Booking Assistant)	member_id	200 { eligible: bool, reason?, membership_id?, seats_left }	—	bookings:read	404	P0
`POST /bookings/{id}/promote`	Manual waitlist promotion	—	200 booking confirmed	seat available; member eligible	bookings:on_behalf	409 BOOKING_CLASS_FULL	P1

3.7 Attendance (B2B-06)

ERP methods: attendance.check_in and attendance.mark_attendance. One record per member per session (BR-ATT-04). Corrections are immutable child records with five audit fields (BR-ATT-03). Event: attendance.recorded.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /attendance/check-in`	Front-desk check-in (single member)	{ session_id, member_id, method: front_desk }	201 { id, status: present, checked_in_at }	member has booking or drop-in allowed by config; session open window	attendance:checkin	409 ATTENDANCE_DUPLICATE · 422 ATTENDANCE_NOT_ELIGIBLE · 409 SESSION_NOT_OPEN	P0
`POST /attendance/mark`	Trainer bulk marking for an assigned session	{ session_id, entries:[{ member_id, status: present|absent|late }] }	200 { results:[{ member_id, ok, error? }] }	trainer must be assigned (BR-ATT-05); ≤ 200 entries; partial success reported per row	attendance:mark	403 NOT_ASSIGNED · 409 ATTENDANCE_DUPLICATE	P0
`POST /attendance/{id}/correct`	Audited correction	{ new_status, reason }	200 { id, previous_status, new_status, corrected_by, corrected_at, reason }	reason ≥ 5 chars required; immutable correction row; original preserved	attendance:correct	422 · 403 · 404	P0
`GET /attendance`	List/search attendance	filter[session_id], filter[member_id], from, to, filter[method]	200 [record + corrections[]]	Trainer auto-scoped	attendance:read	422	P0
`GET /attendance/sessions/{id}/summary`	Session attendance completeness	—	200 { booked, present, absent, unmarked }	—	attendance:read	404	P1

3.8 Events (B2B-08)

ERP method: event.register (capacity, window, audience checks in one transaction). Categories come from config.event_categories (D-03). Events: event.registered, event.registration.confirmed. Paid registrations hold a seat until the configured hold time (B2B-EVT-02).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /events · GET /events/{id}`	List/detail	filter[category], filter[status], from, to, filter[audience]	200 event{id,name,category,starts_at,capacity,registered,is_paid,price,audience,registration_window}	—	events:read	—	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /events	Create event	{ name, category_code, starts_at, ends_at, capacity, registration_opens_at, registration_closes_at, is_paid, price?, audience: public|members_only|prospects, trainer_id? }	201 event	capacity ≥ 1; window before start; price > 0 when paid; category exists	events:create	422 · 404 CATEGORY_NOT_FOUND	P0
PATCH /events/{id} · POST /events/{id}/cancel	Edit/cancel event	partial + If-Match / { reason }	200 event	capacity ≥ confirmed registrations	events:update	409 CAPACITY_BELOW_BOOKINGS · 412	P0
POST /events/{id}/registrations	Register a member or external registrant (staff-assisted)	{ member_id? , registrant{name,phone,email}?, consent{whatsapp} }	201 { id, status: confirmed|pending_payment, hold_expires_at?, payment_required, payment_id? }	one of member_id/registrant; seat + window + audience; external registrant becomes lead/registrant	events:register_others	409 EVENT_FULL · 409 EVENT_REGISTRATION_CLOSED · 403 EVENT_AUDIENCE_RESTRICTED · 409 DUPLICATE	P0
GET /events/{id}/registrations	Registrant list	filter[status]	200 [registration]	—	events:read	404	P0
POST /events/{id}/registrations/{rid}/cancel	Cancel registration	{ reason }	200	—	events:register_others	409	P1

3.9 Payments (B2B-09)

Only a verified server-side Razorpay confirmation marks a payment Paid (BR-PAY-02). ERP method: payment.record_payment (manual) and payment.confirm_payment (from webhook, §11–§12). Event: payment.confirmed / payment.failed. Statuses: created, pending, paid, failed, refunded.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /payments	List payments	filter[status], filter[purpose], filter[member_id], filter[provider], from, to, q	200 [payment{id (PAY-NNNNN), provider, provider_ref, amount, currency, status, purpose, created_at}]	—	payments:read (Trainer denied)	403	P0
POST /payments/orders	Initiate online payment (staff-assisted)	{ purpose: membership|event, target_id, member_id }	201 { payment_id, razorpay_order_id, amount, currency, key_id }	target exists and is payable; amount server-derived (client amount ignored)	payments:collect	409 ALREADY_PAID · 502 PAYMENT_GATEWAY_ERROR	P0
POST /payments/manual	Record cash/offline payment	{ member_id, purpose, target_id, amount, method: cash|upi_manual|card_pos|bank_transfer, reference }	201 payment (status paid, source manual)	actor + reference required (B2B-PAY-02); amount > 0 and equals due unless partial allowed by config	payments:record	422 · 409 ALREADY_PAID	P0
GET /payments/{id}	Payment detail with timeline	—	200 payment + events[]	—	payments:read	404	P0
POST /payments/{id}/verify-status	Force gateway status fetch for stale Pending (reconciliation helper)	—	200 { status }	payment pending > 15 min	payments:reconcile	502 · 409	P1
POST /payments/{id}/refund	Refund (partial/full)	{ amount, reason }	202 { refund_id, status: processing }	permission + reason; amount ≤ paid − refunded; audited (BR-PAY-06)	payments:refund	422 REFUND_EXCEEDS_PAID · 502	P2

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /payments/reconciliation	Unmatched/failed payments	from, to, filter[type]	200 [finding]	—	payments:reconcile	—	P1

3.10 Reports (B2B-10)

All reports come from the Reporting service used by the AI Assistant (identical figures for same metric/period — B2B-RPT-03). Results are permission- and scope-filtered. Exports (P2) apply the same filters.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /reports/{report_code}	Run a core report. report_code ∈ active_members, revenue_by_plan, occupancy, attendance_trend, renewal_pipeline, low_attendance_classes, memberships_expiring	from, to, group_by, filter[…], studio_id	200 { report_code, period, columns[], rows[], totals, generated_at }	date range ≤ 366 d; group_by whitelisted; totals reconcile with module data	reports:read (financial codes need reports:financial)	404 REPORT_NOT_FOUND · 403 · 422	P0
GET /reports	Report catalogue available to caller	—	200 [{ code, name, permission }]	—	reports:read	—	P0
POST /reports/{report_code}/export	CSV/PDF export job	{ format: csv|pdf, …same filters }	202 { job_id }; GET /jobs/{id} → download_url (signed, 10 min)	same filters/permissions	reports:export	422 · 403	P2
GET /reports/kpi-snapshots	Trend from reporting.kpi_snapshots	kpi_code, from, to	200 [{ date, value }]	—	reports:read	—	P1

3.11 Staff (B2B-11)

Staff identity lives in iam.users (App DB); trainer business record in ERP (Wellness Staff) — linked via iam.user_erp_links. Deactivated users cannot authenticate; history keeps attribution.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /staff	List staff	filter[role], filter[is_active], q	200 [staff{id,name,email,roles[],scope,is_active,erp_ref}]	—	staff:read	—	P0
POST /staff	Create staff and send invite	{ full_name, email, phone?, role_codes[], scope{studio_ids,branch_ids}, trainer_profile? }	201 { id, invite_expires_at }	email unique per tenant; role assignable by caller; trainer requires trainer_profile	staff:create	409 CONFLICT_DUPLICATE · 403 · 422	P0
PATCH /staff/{id}	Update details/scope/trainer profile	partial + If-Match	200 staff	cannot remove own Owner role	staff:update	409 LAST_OWNER · 412	P0
POST /staff/{id}/deactivate · /reactivate	Deactivate (blocks login, revokes sessions) / reactivate	{ reason }	200 { is_active }	sessions revoked; future assignments flagged	staff:deactivate	409 STAFF_HAS_ASSIGNED_SESSIONS (warning with reassign option)	P0
POST /staff/{id}/trainer-assignments	Assign trainer to class types/sessions	{ class_type_ids[], session_ids[] }	200 assignments	trainer active	staff:assign	404 · 422	P0
GET /staff/{id}/audit	Staff activity	cursor	200 [audit rows]	—	audit:read	403	P1

3.12 Notifications (B2B-13)

Delivery records live in notify.* (D-01). Templates/timings are tenant configuration (config.notification_templates / notification_rules). Failed messages are visible to admins and never reverse transactions (B2B-NTF-01).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /notifications/messages`	Delivery log (mock and live)	cursor, filter[status]=queued|mock_sent|sent|delivered|read|failed, filter[event_type], filter[recipient_ref], from, to	200 [{ id, event_type, template_code, recipient_masked, channel, status, provider_message_id, error_code, created_at }]	recipient phone masked except last 4	notifications:read	—	P0
`GET /notifications/messages/{id}`	Message detail with delivery events timeline	—	200 message + delivery_events[] + payload_variables	—	notifications:read	404	P0
`GET /notifications/templates · PATCH /notifications/templates/{id}`	View/edit template body, variables, timing	{ body?, is_active? }	200 template (approval_status pending until Meta-approved)	declared variables only; approved templates only sent (BR-NTF-05)	notifications:configure	422 UNDECLARED_VARIABLE	P0
`GET/PUT /notifications/rules`	Trigger mapping and reminder offsets	{ rules:[{ event_type, template_id, offset_minutes, anchor, is_enabled, sequence_no }] }	200 rules	unique (event_type, sequence_no); anchor enum	notifications:configure	409 · 422	P0
`GET /notifications/alerts`	Admin alerts (failed workflows, template rejections)	filter[status]=open	200 [alert]	—	notifications:read	—	P0
`POST /notifications/messages/{id}/retry`	Retry failed message (bounded, dedup)	—	202 { status: queued }	status failed; attempts < max; consent still valid	notifications:retry	409 RETRY_NOT_ALLOWED	P1
`GET/PUT /notifications/mode`	Mock/live status (read-only in UI; switch by config)	—	200 { whatsapp: mock|live }	PUT is environment-controlled, not exposed in Phase 1	notifications:configure	403	P0

3.13 Settings — Configuration Center (B2B-12)

App DB (config.*) for branding, templates, timings, flags, event categories; ERP for plans and class types (§3.4, §3.5). Every change is audited and invalidates the config cache; effective without deployment. Secrets are write-only.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /settings · PATCH /settings`	Tenant settings (default_capacity=30, cancellation_cutoff_minutes=240, time zone, currency INR, waitlist enabled)	{ key:value… }	200 settings	capacity 1–500; cutoff 0–10080 min; audited before/after	settings:update (Owner; Admin limited)	422 · 412	P0
`GET/PATCH /settings/branding · POST /settings/branding/assets`	Studio profile, logo, colours, contacts	{ name, colours{primary,secondary}, contact{…} } / multipart logo	200 branding (theme tokens)	hex colours; image ≤ 2 MB PNG/JPG/SVG; scan + resize	settings:branding	413 · 415 · 422	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET/POST/PATCH /settings/event-categories	Event categories (Workshop, Internal Studio Event, Public Event, Special Session; configurable)	{ code, name, is_active }	200/201	code unique; used categories archived	settings:update	409	P0
GET /settings/feature-flags	Effective flags by tier/tenant (read-only for tenant)	—	200 [{ flag, enabled, source: tier|override }]	—	settings:read	—	P0
GET/PUT /settings/faq	Approved FAQ knowledge for Enquiry Assistant	{ entries:[{ question, answer, status }] }	200	only status=approved is retrievable	settings:update	422	P1
GET/PUT /settings/integrations	Integration credentials (write-only secret_ref)	{ provider, credentials{…} }	200 { provider, configured:bool, last_updated }	secrets never returned; stored in secret store	integrations:manage (Owner)	403	P2

3.14 Tenant Management (B2B-14 — platform-operator side)

Onboarding creates tenant → first studio → first branch → Owner user → default configuration (B2B-TEN-02); seeded/assisted in Phase 1. Platform Administrator has no routine access to tenant business data (PRD §7.3).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /platform/tenants	Create tenant with Owner (onboarding routine)	{ name, slug, tier: basic|premium|ultra, owner{name,email,phone}, first_studio{name}, first_branch{name,timezone} }	201 { tenant_id, studio_id, branch_id, owner_invite_expires_at, erp_route }	slug subdomain-safe & unique; ERP Company created and route recorded; idempotent	platform:tenants:create	409 TENANT_SLUG_TAKEN · 502 ERP_PROVISIONING_FAILED (compensating rollback)	P0
GET /platform/tenants · GET /platform/tenants/{id}	List/detail (no business data)	filter[tier], filter[status]	200 tenant summary	—	platform:tenants:read	—	P0
PATCH /platform/tenants/{id}/tier	Change tier and entitlements	{ tier }	200 { tier, flags[] }	audited; entitlements effective immediately	platform:tenants:update	422	P1
PUT /platform/tenants/{id}/feature-flags/{flag}	Override a feature flag (e.g. ai_assistant, whatsapp_automation)	{ enabled }	200	flag exists	platform:tenants:update	422	P0
POST /platform/tenants/{id}/suspend · /reactivate	Suspend tenant access	{ reason }	200	sessions revoked	platform:tenants:update	409	P1
GET /b2b/studios · POST /b2b/studios · GET /b2b/branches · POST /b2b/branches	Studio/branch structure inside a tenant	{ name, address, timezone }	200/201	branch belongs to tenant studio	tenants:manage (Owner)	422	P1
POST /platform/tenants/{id}/billing	Self-service tier billing	—	—	Phase 2	—	—	P2

4. B2C APIs (Member Portal and Public Site)

Base path `/api/v1/b2c` (authenticated Member) and `/api/v1/public` (Visitor, no login). The API derives member identity from the session (erp_ref claim) and **never accepts a member_id from the client**; a request naming another member's record returns 404. Structure, envelope, errors and headers are identical to §3. Members cannot see other members' data (PRD §7.2).

4.1 Dashboard (B2C-01)

Member summary service; data comes from ERP at request time (status matches ERP). Renewal CTA when ≤ 7 days to expiry.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /dashboard`	Membership status/validity, upcoming bookings, next class, recent attendance, renewal prompt, event highlights	—	200 { membership{plan,status,start_date,end_date,days_left,renewal_cta:bool}, upcoming_bookings[], next_class, recent_attendance[], events[] }	—	self (member)	404 MEMBER_NOT_LINKED	P0

4.2 Memberships (B2C-02)

Purchase creates a Pending membership until verified payment and any required approval (payment and approval are independent — BR-MSP-02).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /membership-plans`	Active plans with tenant-configured price/validity	—	200 [plan{id,name,price,validity_days,allowed_class_types[],benefits}]	only is_active plans	self	—	P0
`GET /memberships`	Own membership history and current status	page	200 [membership]	—	self	—	P0
`POST /memberships/purchase`	Buy or renew a plan; returns checkout payload	{ plan_id, renew_membership_id? }	201 { membership_id, status: pending_approval|pending_payment, payment{ payment_id, razorpay_order_id, amount, currency, key_id } }	plan active; no overlapping active membership unless renewal; amount server-derived; Idempotency-Key required	self	409 MEMBERSHIP_OVERLAP · 404 PLAN_NOT_FOUND · 502 PAYMENT_GATEWAY_ERROR	P0
`POST /memberships/{id}/freeze-request`	Request freeze (approval workflow)	{ from, to, reason }	202	—	self	—	P2

4.3 Bookings (B2C-03)

Results reflect the authoritative ERP outcome, never cached state (B2C-BKG-01). Same ERP method and events as §3.6.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /class-sessions`	Live schedule with per-session eligibility and availability	from, to, filter[class_type_id], filter[trainer_id], filter[time_of_day]	200 [session{…, seats_left, my_booking_status, eligible, ineligible_reason?}]	range ≤ 31 d	self	422	P0

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /bookings	Book a session	{ session_id, join_waitlist_if_full? }	201 { id, status: confirmed|waitlisted, waitlist_position? }	member identity from token; eligibility, limit, duplicate, capacity validated in ERP; Idempotency-Key required	self	409 BOOKING_CLASS_FULL · 409 BOOKING_DUPLICATE · 422 BOOKING_NOT_ELIGIBLE (reason) · 503 ERP_UNAVAILABLE	P0
GET /bookings	Upcoming and past bookings	filter[when]=upcoming|past, page	200 [booking]	—	self	—	P0
POST /bookings/{id}/cancel	Cancel within cutoff window	—	200 { status: cancelled, cutoff_at }	inside cutoff → rejected (default 4 h)	self (own)	409 BOOKING_CUTOFF_PASSED · 404	P0
DELETE /bookings/{id}/waitlist	Leave waitlist	—	204	status waitlisted	self	404 · 409	P0

4.4 Attendance (B2C-04)

ERP method attendance.check_in (method=qr). Duplicate check-ins rejected with a clear message.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /attendance/check-in	QR check-in	{ qr_payload }	201 { id, session, status: present, checked_in_at }	QR signature + expiry; session open window; eligibility; not already checked in	self	422 QR_INVALID · 410 QR_EXPIRED · 409 ATTENDANCE_DUPLICATE · 422 ATTENDANCE_NOT_ELIGIBLE	P0
GET /attendance	History including admin corrections	from, to, page	200 [record{…, corrected:bool, corrections[]}]	—	self	—	P0
GET /attendance/summary	Monthly summary	month	200 { attended, missed, streak_days }	—	self	—	P2

4.5 Events (B2C-05)

Members-only events are hidden from non-members; paid registration is confirmed only after verified payment; unpaid hold expires after the configured hold time.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /events	Audience-filtered events	from, to, filter[category]	200 [event]	audience filter applied server-side	self	—	P0
POST /events/{id}/registrations	Register (and pay if required)	{ consent{whatsapp:bool} }	201 { id, status: confirmed|pending_payment, hold_expires_at?, payment? }	window, capacity, audience	self	409 EVENT_FULL · 409 EVENT_REGISTRATION_CLOSED · 403 EVENT_AUDIENCE_RESTRICTED · 409 DUPLICATE	P0
GET /events/registrations	Own registrations	filter[status]	200 [registration]	—	self	—	P0
DELETE /events/registrations/{id}	Cancel own registration	—	204	before event start	self	409	P1

4.6 Payments (B2C-06)

Client-side Razorpay success is never trusted; status changes only from the verified webhook or server-side status fetch (BR-PAY-02). Pending/failed never render as paid.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`POST /payments/orders`	Create payment order for an existing payable (membership / event registration)	{ purpose, target_id }	201 { payment_id, razorpay_order_id, amount, currency, key_id }	target belongs to caller; unpaid; amount server-derived	self	409 ALREADY_PAID · 502 PAYMENT_GATEWAY_ERROR	P0
`POST /payments/{id}/client-callback`	Optional: post checkout result for UX polling only (does NOT change state)	{ razorpay_payment_id, razorpay_order_id, razorpay_signature }	202 { status: pending }	signature verified server-side then queued for authoritative confirmation; response is never 'paid' by itself	self	422 SIGNATURE_INVALID	P0
`GET /payments/{id}`	Poll payment status	—	200 { status: created|pending|paid|failed|refunded, updated_at }	—	self	404	P0
`GET /payments`	Own payment history	page, filter[status]	200 [payment]	—	self	—	P0
`GET /payments/{id}/receipt`	Receipt/invoice PDF	—	200 signed url	Phase 2	self	—	P2

4.7 Profile (B2C-07)

Members edit only permitted fields; a login-method change re-verifies identity via OTP.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /profile`	Own details	—	200 profile	—	self	—	P0
`PATCH /profile`	Update permitted fields (name, DOB, gender, emergency contact, photo, address)	partial + If-Match	200 profile	email/phone/status excluded; unknown fields → 422 FIELD_NOT_EDITABLE	self	412 · 422	P0
`POST /profile/phone/change (+ /verify)`	Change phone with OTP re-verification	{ new_phone } / { challenge_id, otp }	202 / 200	OTP on new number; duplicate check (never creates duplicate member)	self	409 CONFLICT_DUPLICATE · 401 OTP_INVALID	P0
`POST /profile/google/link · DELETE /profile/google`	Link/unlink Google	{ id_token }	200	verified phone present	self	409 ACCOUNT_LINK_REQUIRED	P1
`GET/PUT /profile/communication-preferences`	Consent and channel preferences	{ whatsapp_marketing:bool, whatsapp_reminders:bool, email:bool }	200	transactional messages not switchable off; recorded in notify.communication_consents (append-only)	self	422	P1

4.8 Notifications (B2C-08)

Phase 1 notifications are WhatsApp only (passive). The in-app centre is Phase 2.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
`GET /notifications`	In-app notification history	cursor, filter[unread]	200 [notification]	—	self	—	P2

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
POST /notifications/{id}/read · /read-all	Mark read	—	204	—	self	—	P2
GET /notifications/whatsapp-history	Read-only list of WhatsApp messages sent to me (status)	cursor	200 [{ template, status, sent_at }]	own recipient only	self	—	P1

4.9 Recommendations (B2C-09)

Rules select the candidate set (eligible + available); LLM ranking/wording optional; no health claims (BR-AI-09). Booking always goes through POST /bookings. Also see §6.4.

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /recommendations	Rules-constrained suggestions with reasons	goal?, level?, limit≤10	200 [{ session, reason, score, book_href }]	goal/level enums from config; all results eligible and available at time of response	self (feature ai_recommendations)	403 FEATURE_NOT_ENTITLED	P0
POST /recommendations/{id}/feedback	Accepted / dismissed signal (acceptance-rate KPI)	{ action: accepted|dismissed }	204	—	self	404	P1

4.10 Public (Visitor) APIs — no login, tenant from host

Rate-limited and bot-protected (§13.8). Only public data and lead creation. Consent is mandatory before follow-up (BR-CRM-04).

Endpoint	Purpose	Request	Response	Validation	Permission	Errors	Pri
GET /public/tenant	Branding/theme, contact details, enabled features	—	200 { name, logo_url, colours, contact, features }	—	public	404 TENANT_NOT_FOUND	P0
GET /public/class-types · /public/class-sessions	Public schedule (SSR/ISR)	from, to, filter[class_type_id]	200 list	public-visible only	public	—	P0
GET /public/plans · /public/events · /public/events/{id}	Public plans and non-members-only events	—	200 list/detail	members-only events excluded	public	—	P0
POST /public/enquiries	Landing/ad form → Lead or Workshop Registrant + confirmation	{ name, phone, email?, interest?, source, campaign?, consent{whatsapp:true}, event_id?, captcha_token }	201 { lead_id (LEAD-NNNNN), message }	captcha verified; consent true required for messaging; duplicate flagged/merged (BR-CRM-03); never creates a Member	public	422 · 429 · 409 (flagged duplicate returns 201 with duplicate_flag)	P0
POST /public/events/{id}/registrations	Public workshop registration without account	{ name, phone, email?, consent, captcha_token }	201 { registration_id, status, payment? }	audience=public/prospects; window; capacity; paid → payment order	public	409 EVENT_FULL · 409 EVENT_REGISTRATION_CLOSED	P0
POST /public/payments/orders	Payment order for an external registrant	{ registration_id }	201 { razorpay_order_id, amount, key_id }	registration pending payment	public (registration token)	409 · 502	P0

5. ERP Integrations (ERPNext / Frappe)

ERP is the system of record; there is **no bidirectional entity sync** (Architecture §7.3). All ERP-owned reads/writes are synchronous call-throughs via the NestJS ERP adapter to the custom app `wellness_studio` on Frappe Cloud. Only IDs are stored in the App DB. ERPNext core is untouched (BR-GEN-01).

5.1 Adapter transport contract

Aspect	Contract
Transport	HTTPS to the Frappe site resolved from platform.tenant_erp_routes (tenant → site/company). Whitelisted methods: POST /api/method/wellness_studio.api.<module>.<method>; simple reads: GET /api/resource/<DocType>.
Identity	Restricted API key/secret of a dedicated ERP integration user (never Administrator) held in the secret store (NFR-SEC-06).
Request headers	X-WS-Tenant (adapter-injected, never from caller), X-Correlation-ID, X-Idempotency-Key (writes), X-WS-Contract-Version, X-WS-Actor (user UUID/type for ERP audit).
Timeouts	Reads 5 s; writes 8 s; report aggregates 15 s. Circuit breaker: opens after 5 consecutive failures/30 s, half-open probe every 15 s.
Response shape	{ ok: true, data } or { ok: false, code, reason, message } — the adapter maps code to application error codes (§13.6). Rule failures return structured reasons (full, ineligible, duplicate).
Version check	On start-up the adapter calls wellness_studio.api.system.contract_version and refuses to serve ERP routes on incompatible major version (ops.erp_contract_versions).
Safe retry	Retries only for safe reads (GET, idempotent read methods): 2 retries, 200 ms/800 ms + jitter. Writes are never blindly retried; a timed-out write is resolved by querying the idempotency key (below).

5.2 Integration catalogue

Direction: Push = API → ERP write; Pull = API → ERP read; Callback = ERP → API. Master Data = slow-changing reference data; Transaction = business events.

#	Integration	Type	Purpose	Trigger	Request flow	Response flow	Sync strategy	Retry	Failure handling
E1	Tenant provisioning (ERPNext Company + Wellness Tenant/Studio/Branch Scope mirrors)	Master data · Push	Create ERP scope for a new tenant (D-02, D-04).	POST /platform/tenants	Onboarding service → adapter → wellness_studio.api.tenant.provision(tenant_id, slug, company_name, studio, branch) with idempotency key = tenant_id.	{ ok, company, site }; route saved in platform.tenant_erp_routes.	One-time, transactional saga; ERP mirror is read-only, written only by this routine.	Idempotent re-run up to 3×.	Compensating rollback of App-DB rows; 502 ERP_PROVISIONING_FAILED; platform alert.
E2	Membership plans & class types (config to ERP)	Master data · Push/Pull	Business configuration lives in ERP; App DB holds no copy.	Settings/Plans and Class-type endpoints	Adapter → resource create/update (Membership Plan, Class Type, Plan Class Entitlement).	ERP document + version; API returns ERP representation.	Synchronous call-through; short-TTL (≤ 60 s) read cache for plan lists only, invalidated on write.	Reads: 2 retries. Writes: none automatic.	503 ERP_UNAVAILABLE; UI shows failure; no optimistic state.
E3	Member create/update/archive	Master data · Push	System-of-record member with duplicate control.	POST/PATCH /members, AI Create Member, lead conversion, OTP registration linking	member.create_member(payload) with duplicate check on email/phone inside ERP.	MEM-NNNNN or code duplicate/validation.	Synchronous; iam.user_erp_links stores user↔MEM- mapping.	None on writes; on timeout the adapter queries by idempotency key before reporting.	409 CONFLICT_DUPLICATE mapped verbatim; timeouts → 504 ERP_TIMEOUT with 'outcome unknown' state and reconciliation entry.
E4	Lead capture & conversion	Transaction · Push	Lead/Workshop Registrant creation; explicit conversion.	POST /public/enquiries, /leads, /leads/{id}/convert	lead.create_lead / lead.convert_lead.	LEAD- ID; conversion returns member_id + preserved source.	Synchronous; outbox lead.created after ERP success.	None on writes.	Duplicate flagged/merged (BR-CRM-03); ERP down → public form returns 503 with retry hint; no lead silently dropped (client shows retry).

#	Integration	Type	Purpose	Trigger	Request flow	Response flow	Sync strategy	Retry	Failure handling
E5	Membership lifecycle	Transaction · Push + Callback	Assign, approve/ activate, suspend, renew, expire.	Membership endpoints; pay ment.confirmed; ERP scheduler (expiry)	membership.activate_membersh ip / suspend_membership; expiry by scheduler → ERP callback.	State + timestamps; callbacks POST /int ernal/events/mem bership.expiring|e xpired.	ERP owns state machine; API emits outbox event after ERP result; scheduler events audited then outboxed.	Callback delivery retried by ERP (exponential, 6×/24 h).	Missed callback caught by nightly reconciliation (ops.reconciliation_*) and n8n 'due' query.
E6	Class sessions & schedules	Transaction · Push/Pull	Session creation, capacity edit, roster.	Classes endpoints; schedule expander	Adapter → Class Schedule/Session resources; capacity change checked against confirmed bookings in ERP.	Session docs with counts.	Synchronous; never cached for availability.	Reads 2×.	CAPACITY_BELOW_BOOKINGS returned as 409.
E7	Booking (atomic)	Transaction · Push	Exactly-one-win ner seat booking; waitlist promotion.	POST /bookings; cancel	booking.create_booking(session, member, actor, idempotency_key) — row lock, eligibility, limit, duplicate, capacity; cancel_booking → promote_waitlist.	confirmed | waitlisted | rejected(reason).	Synchronous; authoritative; events booking.confirmed / waitlist.promoted emitted after commit.	No auto-retry; idempotency key makes client retry safe.	Timeout → status query by key; unknown outcome is shown as 'could not confirm — check bookings', never assumed.
E8	Attendance	Transaction · Push	Check-in, trainer marking, immutable corrections.	Attendance endpoints	attendance.check_in / mark_attendance; correction as child record.	record or duplicate/eligibility code.	Synchronous.	None on writes.	ATTENDANCE_DUPLICATE etc. mapped 1:1.
E9	Event registration	Transaction · Push	Capacity/windo w/audience checks; payment hold.	Event registration endpoints	event.register(event, registrant, actor); hold timer handled by ERP scheduler.	confirmed | pendin g_payment(hold_e xpires_at).	Synchronous; hold expiry → ERP callback event.registration.expired.	Callback retried.	Expired hold releases seat; notification via outbox.
E10	Payment posting	Transaction · Push	Post verified Razorpay payment / manual payment to ERP payment record and dependent state.	Verified webhook payme nt.captured; POST /payments /manual	payment.confirm_payment(orde r_id, razorpay_payment_id, amount, verified_at) or payment.record_payment(…).	Payment Paid + dependent membe rship/registration updated.	Executed inside the webhook handler after signature verification; idempotent on razorpay_payment_id.	Bounded queue retry (5×, back-off 30 s→30 min) if ERP unavailable; payment stays Pending.	Never mark Paid without ERP confirmation; open reconciliation finding after 15 min pending.
E11	Report aggregates	Pull	KPI and report data for Dashboard, Reports, Assistant.	GET /dashboard, /reports/*, AI tools	Adapter → wellness_studio.api.re port.<code>(filters, scope).	Aggregated rows.	Pull on demand; ≤ 60 s cache on dashboard only; scheduled aggregates for heavy reports (reporting.kpi_snapshots).	Reads 2×.	Partial tiles allowed: failed tile returns error object, others render.
E12	ERP event callbacks (Callback)	Transaction · Callback	ERP-originated events into the outbox path.	Frappe scheduler / doc_events	POST /internal/events/{members hip.expiring|membership.expire d|session.starting|session.full|ev ent.registration.expired} with HMAC signature + timestamp + event_id.	200 { accepted } (duplicates return 200 no-op).	API validates, audits, writes outbox (same path as app events).	ERP retries 6× with exponential back-off.	401 on bad signature (discarded, security event); dead-letter after retries visible in ERP error log.

#	Integration	Type	Purpose	Trigger	Request flow	Response flow	Sync strategy	Retry	Failure handling
E13	Reconciliation (nightly)	Pull	Detect orphan references, unresolved Pending payments, events without delivery outcome.	Worker schedule 02:00 IST	Adapter bulk-reads IDs by tenant; compares with App DB references.	ops.reconciliation_runs/findings.	Read-only; reports to admins; no auto-fix of ERP data.	Run retried next window.	Findings surface in Notifications alerts.
E14	Trainer / staff business record	Master data · Push/Pull	Trainer profile in ERP linked to iam user.	POST /staff, trainer assignment	Wellness Staff resource create/update.	STF- ID stored in iam.user_erp_links.	Synchronous.	Reads 2×.	Staff creation rolled back if ERP write fails.

5.3 Frappe (custom app) responsibilities and ERP → API callback contract

ERP artefact	Contract
Whitelisted methods (write)	booking.create_booking · booking.cancel_booking · booking.promote_waitlist · attendance.check_in · attendance.mark_attendance · membership.activate_membership · membership.expire_membership · membership.suspend_membership · member.create_member · payment.record_payment · payment.confirm_payment · event.register · lead.create_lead · lead.convert_lead (Module Spec §9.2). Plus contract-level helpers defined by this document: tenant.provision, system.contract_version, report.<code>.
Structured failure	{ ok:false, code: BOOKING_CLASS_FULL|BOOKING_NOT_ELIGIBLE|BOOKING_DUPLICATE|CONFLICT_DUPLICATE|ATTENDANCE_DUPLICATE|CAPACITY_BELOW_BOOKINGS|INVALID_STATE_TRANSITION|VALIDATION_FAILED, reason, message }.
Callback envelope	{ event_id, type, schema_version, tenant_id, occurred_at, subject:{type,id}, data{…}, correlation_id } with headers X-WS-Signature = HMAC-SHA256(secret, timestamp + '.' + body) and X-WS-Timestamp; replay window ±5 min; event_id de-duplicated in ops.webhook_inbox.
Tenant scoping	permission_query_conditions on every Wellness DocType via get_tenant_condition; adapter always injects tenant; cross-tenant test suite runs in CI (Architecture S2).
ERP unavailable	Adapter returns ERP_UNAVAILABLE (503); nothing cached is presented as confirmed (NFR-AVL-03); outbox events are not created for failed writes.

5.4 ERPNext vs Frappe — what each integration touches

Layer	Used for	Integration surface	Phase
Frappe framework (custom app wellness_studio)	Wellness DocTypes (Member, Membership, Plan, Class Type/Schedule/Session, Booking, Attendance + Correction, Event + Registration, Lead + Activity, Payment Record, Wellness Staff, tenant scope mirrors), whitelisted atomic methods, permission hooks, scheduler, doc_events → signed callbacks	/api/method/wellness_studio.api.* and /api/resource/<Wellness DocType>; callbacks to /internal/events/*	P0
ERPNext core (upstream, unmodified)	Company (one per tenant), Customer/Sales Invoice/Payment Entry, accounting, India GST compliance; reused where the structures fit	Read/link only in Phase 1 (Payment Record keeps optional erp_invoice_ref); posting via custom-app methods only	P2 (invoicing, GST, reconciliation)
Frappe Cloud platform	Managed bench hosting the custom app; backups; site per environment	Not an API surface; deployment via GitHub branch → bench (Architecture S1)	P0

6. AI Service APIs

Base path `/api/v1/ai` (staff), `/api/v1/b2c/ai` (member) and `/api/v1/public/ai` (visitor). AI is a module inside the NestJS API (AD08): endpoints call the LLM gateway, which calls the **tool registry**, whose tools are thin wrappers over the same domain services as the REST controllers. The model never reaches the database or ERP (BR-AI-01). All AI endpoints are gated by tier and feature flag (BR-AI-11; 403 FEATURE_NOT_ENTITLED), require an Idempotency-Key on state-changing calls, and write an ai.ai_tool_call_audit row for every model-initiated tool call (NFR-AUD-03).

6.1 Common AI contract

Aspect	Contract
Message response	{ session_id, turn_no, type: answer|question|confirmation|action_result|out_of_scope|handoff, text, data?, pending_action?{ id, summary, arguments_preview, expires_at }, tools_used[{name,status}], correlation_id }. Answers derive only from tool results (Architecture §8.1). Unsupported questions return type=out_of_scope with a scope message, HTTP 200 — not a guess.
Two-phase confirmation	Write actions create a row in ai.ai_pending_actions with arguments frozen (hash stored). Only POST …/confirm executes those stored arguments through the same service as the UI; the client cannot modify them. Expiry 10 min. Relative dates ('Monday') are resolved by the service layer and shown in the confirmation summary.
True result	After confirm, the response carries the real outcome: { status: succeeded|failed, record_id?, error{code,message} }. Success is never claimed without a success result (BR-AI-05); validation/duplicate errors are relayed verbatim.
Streaming (optional)	Accept: text/event-stream on the messages endpoint streams events token, tool_start, tool_end, final; the final event has the same JSON as the non-streaming body.
Permission inheritance	Tool list is filtered per user, tenant, tier and flag when building the prompt and re-checked at execution (BR-AI-02). Trainer/Member/Visitor principals never receive tools outside their domain.
PII and logging	PII minimised before prompts and logs (NFR-SEC-05): phone/e-mail masked; ERP IDs used as references. Conversation turns are temporary (purge_after) and never become business data except through a governed service (BR-AI-07).
Provider resilience	Per-call timeout 20 s; one bounded retry; if unavailable → 503 AI_UNAVAILABLE with UI 'assistant unavailable' while dashboards remain unaffected. Token/cost quotas per tenant and tier (ai.ai_usage_logs); over quota → 429 AI_QUOTA_EXCEEDED.
Performance targets	Read queries p95 ≤ 8 s (NFR-PRF-04); Enquiry median ≤ 5 s.

6.2 AI Assistant (admin) — B2B-15

Phase 1 tools: memberships_expiring_this_week, low_attendance_classes (read); create_member (write, sensitive, confirmation). Front Desk gets read tools only when the Owner has granted ai:query.

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
`POST /ai/assistant/sessions`	{ locale? }	201 { session_id, expires_at, tools:[names], disclaimer }	Staff JWT; ai:query; tenant-bound; session tied to user and sid	10 sessions/hour/user	403 AI_FEATURE_DISABLED · 429	P0
`POST /ai/assistant/sessions/{id}/messages`	{ message (1–2000 chars), client_msg_id }	Message response (§6.1): answer (e.g. list of expiring memberships identical to dashboard), question for missing mandatory field, or confirmation with pending_action	ai:query (read) / members:create checked again for create_member; input sanitised, prompt-injection filters; tool args schema-validated	30 msg/min/user; 500 msg/day/user; 20 concurrent sessions/tenant	422 VALIDATION_FAILED · 404 AI_SESSION_NOT_FOUND/expired · 429 AI_RATE_LIMITED · 503 AI_UNAVAILABLE · 504 AI_TIMEOUT · 200 out_of_scope	P0
`POST /ai/assistant/actions/{pending_id}/confirm`	{ } (no arguments accepted)	200 { status: succeeded|failed, record_id (e.g. MEM-00043), error?, audit_id }	Only the requesting user; pending action not expired/decided; permission re-checked at execution; Idempotency-Key	10/min/user	410 AI_ACTION_EXPIRED · 409 AI_ACTION_ALREADY_DECIDED · 403 PERMISSION_DENIED · 409 CONFLICT_DUPLICATE (verbatim) · 503 ERP_UNAVAILABLE	P0
`POST /ai/assistant/actions/{pending_id}/reject`	{ reason? }	200 { status: rejected }	Requesting user	10/min/user	410 · 409	P0

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
GET /ai/assistant/sessions/{id}	—	200 { turns:[…], pending_actions[] } (temporary)	Owner of session only; Owner may view others via audit endpoint	60/min	404	P1
DELETE /ai/assistant/sessions/{id}	—	204; conversation purged	Session owner	—	404	P1
GET /ai/audit	cursor, filter[user_id], filter[tool], from, to	200 [{ id, actor, tool, arguments_redacted, decision, approval, result, at }]	audit:read (Owner full, Admin limited)	30/min	403	P0

```
// POST /ai/assistant/sessions/{id}/messages   (Create Member flow, turn 1)
{ "message": "Add Rahul, Premium, starting Monday", "client_msg_id": "7d1e…" }
// 200 — missing mandatory field: asks, never infers (BR-AI-04)
{ "success": true, "data": { "type": "question", "text": "What is Rahul's phone number and email?", "missing_fields": ["phone","email"] } }
// turn 2 → 200 confirmation
{ "type": "confirmation", "text": "Create member Rahul Menon (Premium, start 2026-10-05, Monday)?",
  "pending_action": { "id": "pa_91c…", "tool": "create_member", "summary": "…", "expires_at": "2026-10-05T09:25:00Z" } }
// POST /ai/assistant/actions/pa_91c…/confirm → 200
{ "data": { "status": "succeeded", "record_id": "MEM-00043" } }
// duplicate → 200 { "status": "failed", "error": { "code": "CONFLICT_DUPLICATE", "message": "A member with this phone already exists (MEM-00012)." } }
```

6.3 AI Enquiry Assistant (public)

Runs under a restricted visitor principal: public-read tools plus create_lead only. Answers only from approved knowledge (public classes/schedule/plans/events + config.faq_entries status=approved). No invented prices or promises (BR-AI-08); escalates when confidence or coverage is insufficient.

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
POST /public/ai/enquiry/sessions	{ captcha_token, page_context? }	201 { session_id, greeting, expires_at }	Public; tenant from host; bot protection; session cookie; no PII in URL	5 sessions/hour/IP	429 · 403 CAPTCHA_FAILED	P0
POST /public/ai/enquiry/sessions/{id}/messages	{ message (≤ 1000 chars) }	Message response: grounded answer (with source refs), lead_capture prompt, or handoff	public tools only; output filter blocks unapproved claims; injection filter	10 msg/min/session, 40 msg/session, 60 msg/hour/IP	429 AI_RATE_LIMITED · 503 AI_UNAVAILABLE (fallback text + enquiry form link) · 200 handoff	P0
POST /public/ai/enquiry/sessions/{id}/lead	{ name, phone, email?, consent{whatsapp:true}, interest }	201 { lead_id, message } (calls same CRM service as POST /public/enquiries)	explicit consent captured and recorded; duplicate flagged; captcha proof from session	3 leads/hour/IP	422 · 409 (flagged) · 429	P0
POST /public/ai/enquiry/sessions/{id}/handoff	{ reason?, contact? }	202 { handoff_id }; admin alert raised	Session-bound	3/hour/session	429	P1

6.4 AI Booking Assistant (member)

Phase 1: read-only eligibility check + shortlist with deep links into the standard booking flow. No booking tool exists (booking suggestions never create bookings). In-chat booking (propose/confirm) is Phase 2.

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
POST /b2c/ai/booking/suggest	{ class_type?, day?, time_of_day?, message? }	200 { suggestions:[{ session_id, starts_at, class_type, trainer, seats_left, reason, book_href }], text }	Member JWT; member identity from token; tools: classes.search, bookings.availability, memberships.eligibility (read-only)	20 req/min/user; 200/day	422 · 429 · 503 AI_UNAVAILABLE (fallback: rules-only shortlist, ai=false) · 403 FEATURE_NOT_ENTITLED	P0

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
POST /b2c/ai/booking/sessions/{id}/messages	{ message }	Message response with suggestions[] (chat form)	same; 100% of suggestions eligible & available at time of response	30 msg/min/user	as above	P1
POST /b2c/ai/booking/propose (Phase 2)	{ session_id }	200 { pending_action } → confirm creates booking via POST /bookings service	member confirmation required	10/min	—	P2

6.5 AI Recommendation Assistant (member)

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
POST /b2c/ai/recommendations	{ goal?: flexibility|strength|cardio|stress_relief|..., level?: beginner|intermediate|advanced, explain?: bool }	200 { recommendations:[{ session_id, class_type, rank, reason }], candidate_count }	Member JWT; rules choose candidate set (eligible + available), LLM only ranks/explains within it; no health/medical claims (guardrail check on output)	10 req/min/user; 100/day	422 · 429 · 503 → rules-ordered list without explanation	P0
GET /b2c/recommendations	see §4.9	as §4.9	same	30/min	—	P0
POST /b2c/ai/recommendations/{id}/feedback	{ action }	204	own recommendation	60/min	404	P1

6.6 AI Feedback Assistant

Endpoint	Inputs	Outputs	Security	Rate limits	Error handling	Pri
POST /b2c/feedback	{ session_id (class session attended), rating? 1–5, comment (≤ 1000 chars) }	201 { feedback_id, status: received } — classification is asynchronous	Member JWT; must have attendance record for session; raw comment stored as record; PII scrubbed before LLM call	5 submissions/day/member; 1 per session	409 FEEDBACK_ALREADY_SUBMITTED · 403 NOT_ATTENDED · 422	P1
GET /b2b/ai/feedback	filter[sentiment], filter[topic], filter[class_type_id], from, to, cursor	200 [{ id, comment, rating, sentiment, topic, key_point, session_ref, analyzed_at }]	ai:feedback_read (Owner/Admin); results advisory only — no automatic member action	60/min	403	P1
GET /b2b/ai/feedback/themes	week=YYYY-Www	200 { week, themes:[{ topic, sentiment_mix, count, summary }] }	same	30/min	404 THEMES_NOT_READY	P1
POST /b2b/ai/feedback/{id}/reclassify	{ }	202 { status: queued }	ai:feedback_read; job via worker	10/min	429 · 503	P2
POST /internal/automation/ai/feedback/classify (worker)	{ feedback_id }	200 { sentiment: positive|neutral|negative, topic, key_point, confidence }	service principal; tenant-scoped; not browser-reachable	queue-controlled; ≤ 3 retries, back-off	failed → status=failed, admin alert; raw comment unaffected	P1

7. Future Agent APIs (Phase 2+)

Agents run as background workers (apps/worker) using the same tool registry, LLM gateway, audit and pending-action store as the Assistant (Architecture §9). None is Phase 1: APIs below are specified for contract stability and are feature-flagged off (agent_* flags). Each agent has a **service principal** (iam.service_principals) scoped to one tenant and one domain toolset; when acting for a user, that user's permissions are a ceiling. Agents propose; a human decision (agent.agent_decisions) is required for sensitive actions. Agents never call each other — collaboration is through outbox events.

7.1 Common agent API contract

Class	Endpoint	Contract	Security / errors
Trigger API	POST /api/v1/agents/{agent}/runs	Manual/API trigger: { trigger: manual|schedule|event, scope{…}, params{…}, requested_by? } → 202 { run_id, status: queued }. Event and schedule triggers are created by the worker from outbox events (e.g. lead.created) or cron and use the internal form POST /internal/agents/{agent}/trigger with service credentials.	agents:trigger; feature flag; per-agent quota; 403 FEATURE_NOT_ENTITLED · 409 AGENT_RUN_ACTIVE (dedupe key) · 429
Execution API (internal)	POST /internal/agents/{agent}/runs/{run_id}/execute · POST …/steps	Called by the worker only: executes the plan step by step through governed tools; each tool call writes an audit row (actor_type=ai, initiator=agent). Not exposed to browsers.	service principal; tool allow-list per agent (agent.agent_tool_grants); 403 AGENT_TOOL_DENIED · 504 AGENT_TIMEOUT (max 5 min/run) · retries 2×
Status API	GET /api/v1/agents/{agent}/runs/{run_id} · GET /api/v1/agents/{agent}/runs?filter[status]	200 { run_id, agent, status: queued|running|awaiting_approval|succeeded|failed|cancelled, progress{step,total}, started_at, finished_at, error? }; POST …/cancel to stop.	agents:read; tenant-scoped; 404
Result API	GET /api/v1/agents/{agent}/runs/{run_id}/result	200 agent-specific result (see table); drafts and recommendations only — never a side effect.	agents:read; 409 AGENT_RUN_NOT_FINISHED · 410 RESULT_EXPIRED
Approval API	GET /api/v1/agents/decisions?filter[status]=pending · POST /api/v1/agents/decisions/{id}/approve | /reject	Approval queue (new UI Phase 2). approve → executes the stored action through the governed service and returns the true result; reject records reason. Arguments cannot be changed at approval time.	agents:approve (Owner/Admin per action risk); 409 DECISION_ALREADY_MADE · 410 DECISION_EXPIRED

7.2 Agent catalogue

Agent (phase)	Trigger APIs / events	Execution (internal tools)	Status API	Result API payload	Human approval
CRM Agent (P2)	Event lead.created; schedule stale_leads (daily); POST /agents/crm/runs {lead_id?}	tools: leads.read, leads.history, sources.read, followups.draft (no send tool)	GET /agents/crm/runs/{id}	{ lead_id, score, qualification, next_action, draft_message{template_candidate,text} }	Required before any message is sent (send occurs via approved notification path); score advisory
Operations Agent (P2)	Schedule ops_scan (hourly); events session.full, booking.cancelled, membership.expired; POST /agents/operations/runs	tools: classes.read, bookings.read, attendance.read, memberships.read, alerts.create, schedule.propose	GET /agents/operations/runs/{id}	{ alerts[], tasks[], schedule_proposals[] }	None for alerts; required for schedule changes and member-facing actions
Marketing Agent (P3)	POST /agents/marketing/runs {goal, segment}; schedule renewal_campaign	tools: segments.read, consent.read, campaigns.draft, calendar.read	GET /agents/marketing/runs/{id}	{ audience{definition,count,opted_in}, drafts[{channel,template,text}], performance? }	Mandatory before every send; opt-outs enforced by platform, not agent
Advertising Agent (P3)	POST /agents/advertising/runs {offer|event}; schedule weekly_source_review	tools: lead_sources.read, offers.read, events.read, adcopy.draft (no publish credentials)	GET /agents/advertising/runs/{id}	{ ad_drafts[], audience_suggestions[], landing_page_recs[], performance_analysis }	Mandatory before publication or any spend (BR-AI-10)
Growth Optimization Agent (P3)	POST /agents/growth/runs {period}; monthly schedule	tools: revenue.read, occupancy.read, conversion.read, plans.read	GET /agents/growth/runs/{id}	{ recommendations:[{rank, area, action, rationale, expected_impact}] }	Advisory only; owner acts through normal UI

Agent (phase)	Trigger APIs / events	Execution (internal tools)	Status API	Result API payload	Human approval
Studio Insights Agent (P3)	Schedule weekly_briefing / daily_anomaly; POST /agents/insights/runs	tools: reporting.* (read-only)	GET /agents/insights/runs/{id}	{ briefing{summary,trends[],anomalies[],drivers[]}, forecasts? }	Read-only briefing delivered without approval; derived actions go through other agents' queues
Session Summary Agent (Future)	Event attendance.recorded / session ended; POST /agents/session-summary/runs {session_id|member_id, period}	tools: attendance.read, sessions.read, trainer_notes.read (approved only)	GET /agents/session-summary/runs/{id}	{ audience: member|staff, summary_text, sources[] }	Trainer/admin review before member-facing delivery (default)

Agent runs write agent.agent_tasks / agent_executions / agent_logs; every action is logged with initiator, capability, arguments, decision, approval, result and timestamp (PRD §11.8). Agent output that resembles a business action (send, publish, change schedule) is always converted into an agent_decisions row first.

8. Automation & n8n Integrations

Pattern (AD06/AD07): business transaction commits → audit + outbox row in one PostgreSQL transaction → outbox dispatcher (worker) → signed webhook to n8n → workflow calls the API (never the database/ERP) → shared **send-message** sub-workflow → WhatsApp/mock → delivery status → API. n8n orchestrates; it owns no business state (BR-NTF-01). Workflow definitions are exported JSON in automation/n8n/workflows.

8.1 Event envelope and dispatcher contract

```
// Dispatcher → n8n     POST {N8N_BASE}/webhook/{workflow}    (private network)
// Headers: X-WS-Event-Id, X-WS-Timestamp, X-WS-Signature: sha256=HMAC(secret_per_env, timestamp + "." + body), X-Correlation-ID
{ "event_id": "0b1c…",            // UUID; consumer idempotency key
  "type": "booking.confirmed",
  "schema_version": 1,
  "tenant_id": "2f5e…", "studio_id": "…", "branch_id": "…",
  "occurred_at": "2026-10-05T09:12:44Z",
  "subject": { "type": "Booking", "id": "BKG-00981" },
  "correlation_id": "c1f0a5e2-…",
  "data": { "member_id": "MEM-00042", "session_id": "SES-01234", "starts_at": "2026-10-06T12:30:00Z", "status": "confirmed" } }
// n8n → 200 { "accepted": true } within 5 s (workflow continues asynchronously). Non-2xx / timeout ⇒ dispatcher retry.
```

Rule	Specification
Payload minimisation	Envelope carries IDs and non-sensitive event data only. Recipient phone numbers and message variables are fetched by n8n from the API at send time (prepare call) — never placed in the outbox payload or n8n execution history.
Dispatcher retry	Outbox status pending → dispatching → delivered | failed → dead. Polling with FOR UPDATE SKIP LOCKED. Attempts: 30 s, 2 min, 10 min, 1 h, 6 h, 24 h (6 attempts) with ±20% jitter; then status=dead and an admin alert (notify.admin_alerts). Delivered rows retained 30 days.
Ordering / dedup	At-least-once delivery; consumers idempotent on event_id. Per-subject ordering is best-effort; workflows must tolerate out-of-order (e.g. cancelled before confirmed).
n8n outage	Events queue in outbox; transactions unaffected (NFR-AVL-02). Backlog alert when oldest pending > 10 min.
Replay	POST /internal/automation/outbox/{event_id}/replay (Owner via admin tool, P1) re-queues a dead event; dedupe keys prevent duplicate messages.

8.2 Internal automation API (n8n → NestJS)

Authenticated with service credentials (client-credentials JWT or HMAC-signed request, typ=service, scope automation:*), tenant taken from the event/tenant_id and validated against the credential's tenant allow-list. Not routable from the internet (private network + allow-list).

Endpoint	Purpose	Request	Response	Errors
`POST /internal/automation/messages/prepare`	Resolve template, variables, recipient, consent and dedupe in one call before sending	{ event_id, event_type, template_code, subject{type,id}, recipient_ref (e.g. MEM-00042|LEAD-00012), sequence_no?, tenant_id }	200 { send: true, notification_id, mode: mock|live, provider_template_name, language, to (E.164, one-time), variables:[…ordered] } or { send:false, reason: opted_out|duplicate|template_inactive|template_not_approved|no_phone|feature_disabled }	401 · 404 SUBJECT_NOT_FOUND · 409 DUPLICATE_SEND (also returned as send:false) · 503 ERP_UNAVAILABLE (n8n retries)
`POST /internal/automation/messages/{notification_id}/result`	Report send outcome (accepted by WhatsApp / mock / failed)	{ status: mock_sent|sent|failed, provider_message_id?, error{code,message}?, attempt }	200 { recorded: true }	409 INVALID_STATE_TRANSITION · 404
`GET /internal/automation/due`	'What is due' query for scheduled reminders (tenant-scoped, de-duplicated)	type=class.reminder|event.reminder|membership.expiring|membership.expiry_day, tenant_id, now, lookahead_minutes	200 [{ event_type, subject, recipient_ref, offset_minutes, sequence_no, dedupe_key }]	422 · 401

Endpoint	Purpose	Request	Response	Errors
`POST /internal/automation/executions`	Start/finish a workflow execution record	{ workflow, event_id, status: started|succeeded|failed, n8n_execution_id, error? }	200 { execution_id }	409
`POST /internal/automation/alerts`	Raise admin alert for persistent failure	{ severity, code, message, event_id?, workflow }	201 { alert_id }	—
`POST /internal/events/{type}`	ERP/scheduler callback intake (see §5.2 E12)	signed envelope	200 { accepted }	401 SIGNATURE_INVALID
`POST /internal/automation/events`	n8n-originated domain trigger (e.g. schedule fired 'renewal_due') converted into an outbox event	{ type, subject, data, dedupe_key }	202	409 DUPLICATE_EVENT

8.3 Workflow contracts

Workflow (n8n file)	Trigger events	Payload data (envelope.data)	Steps	Retry logic	Failure handling	Phase
Lead workflows (lead-confirmation.json)	lead.created · lead.followup_due (P2) · lead.converted (stops sequence) · trial.reminder_due (P2)	{ lead_id, source, campaign?, interest, consent_whatsapp }	prepare(template lead_confirmation) → send-message; P2: follow-up sequence timers; cancel on lead.converted	n8n node retry 3× (10 s, 1 min, 5 min) on API 5xx; message-level retry in send-message; dispatcher retry on non-2xx	prepare send:false ⇒ no message and log reason; final failure ⇒ POST alerts; never blocks lead creation	P0 (conf.) / P2
Membership workflows (renewal-reminders.json, payment-received.json)	membership.activated · membership.expiring (7/3/1 d) · membership.expired (expiry day) · membership.pending_approval (admin task)	{ membership_id, member_id, plan_name, end_date, days_left }	Activation notice; schedule 'due' query daily 09:00 IST → per-recipient prepare/send; expiry-day notice; P2 status check + escalation	as above; due query idempotent by dedupe_key (event/sequence/recipient)	duplicate dedupe_key ⇒ skipped; failure ⇒ alert + delivery log failed	P0 reminders · P2 escalation
Booking workflows (booking-confirmation.json, class-reminders.json)	booking.confirmed · booking.waitlisted · booking.cancelled · waitlist.promoted · class.reminder (12 h, 1 h — schedule) · session.cancelled (P1)	{ booking_id, member_id, session_id, starts_at, class_type, status }	Immediate confirmation; reminders via schedule + 'due' query; cancellation message; promotion notice (P2)	as above; reminder not sent if booking cancelled (prepare re-checks status)	cancelled-before-send ⇒ send:false; failures alerted	P0 confirm/remind · P2 promotion
Attendance workflows (attendance-*.json, P2)	attendance.recorded · attendance.missed (session end, booked but absent) · member.inactive_3d/7d (P2)	{ attendance_id, member_id, session_id, method, status }	Optional attendance confirmation / feedback link; missed-session and inactivity nudges	as above	opt-out honoured for non-transactional; failures alerted	P1 (feedback link) / P2
Payment workflows (payment-received.json)	payment.confirmed · payment.failed · payment.refunded (P2)	{ payment_id, purpose, target_id, amount, currency, member_id?|registrant_id? }	Payment-received message; membership-activated notice if approval already granted; failure ⇒ admin exception task	as above; payment.confirmed never re-verified by n8n (source of truth is API)	message failure never reverses payment; alert to admin	P0
Notification workflows (send-message.json, failure-alert.json)	Invoked by every workflow (sub-workflow) · delivery status webhooks from WhatsApp	{ notification_id, mode, provider_template_name, to, variables[] } (from prepare)	mode=mock ⇒ result(status=mock_sent) with would-send record; mode=live ⇒ WhatsApp Cloud API call → result; status callbacks (§9.3) update delivery events	Send retry: 3 attempts, back-off 30 s / 2 min / 10 min on 429/5xx/timeout; 4xx template/recipient errors not retried	Bounded retries then status=failed + alert; template rejection surfaced to admin (code TEMPLATE_REJECTED)	P0

Workflow (n8n file)	Trigger events	Payload data (envelope.data)	Steps	Retry logic	Failure handling	Phase
Event workflows (event-registration.json)	event.registered · event.registration.confirmed · event.registration.expired · event.reminder (24 h/6 h/1 h; workshops 24 h/12 h/1 h — P2)	{ event_id, registration_id, member_id?|registrant_id?, starts_at, category }	Registration confirmation immediately (P0); reminders (P2)	as above	as above	P0 confirm · P2 reminders

8.4 Event catalogue

Event type	Emitted by	After	Consumers
lead.created · lead.converted	CRM service	ERP lead commit	lead workflow, CRM Agent (P2)
membership.pending_approval · membership.activated · membership.expiring · membership.expired · membership.suspended	Memberships service / ERP callback	ERP state commit	membership workflow, Operations Agent (P2)
booking.confirmed · booking.waitlisted · booking.cancelled · waitlist.promoted · session.full · session.starting · session.cancelled	Bookings service / ERP callback	ERP commit	booking workflow, Operations Agent
attendance.recorded · attendance.corrected	Attendance service	ERP commit	attendance workflow, Session Summary Agent (Future)
payment.confirmed · payment.failed · payment.refunded	Payments service (webhook handler)	verified webhook + ERP posting	payment workflow, Memberships service (activation check)
event.registered · event.registration.confirmed · event.registration.expired · event.reminder	Events service / ERP callback	ERP commit	event workflow
notification.failed · workflow.failed	Notification service / n8n	dead-letter	admin alert; outbound webhook (§12)
ai.action.executed · agent.decision.made	AI module	true result recorded	audit, agents

9. WhatsApp Communication Architecture

9.1 Architecture and shared rules

Aspect	Specification
Path	Domain event → outbox → n8n workflow → POST /internal/automation/messages/prepare → send-message sub-workflow → WhatsApp Cloud API (or mock) → status webhook → API (§8, §12.3). The API, not n8n, decides consent, template, variables and dedupe.
Send call	POST https://graph.facebook.com/{version}/{phone_number_id}/messages with Bearer access token from the per-tenant secret reference; body type=template with template name, language and body parameters (Module Spec §9.4). Graph API version is configuration (Module Spec shows v18.0 as reference; confirm current supported version at go-live).
Templates	Approved templates only (BR-NTF-05). Logical templates live in config.notification_templates (code, category transactional|marketing, provider_template_name, declared variables, approval_status). Business-initiated messages are always template messages. Free-form replies are not sent by the platform in Phase 1.
Consent	Transactional messages (confirmations, reminders, payment, renewal) are sent to members with a valid phone on file; lead follow-up and all marketing-type messages require recorded opt-in (notify.communication_consents) and honour opt-outs (BR-CRM-04, BR-NTF-05). Consent is checked in prepare, never in n8n.
Idempotency	dedupe_key = event_id + recipient_ref + template_code + sequence_no; UNIQUE (tenant_id, dedupe_key) in notify.notification_messages; at most one message per event and recipient (BR-NTF-03).
Mock mode (Phase 1)	prepare returns mode=mock; the workflow does not call Meta; result(status=mock_sent) stores a would-send record (recipient, template, variables, time — B2B-NTF-02). Switch to live = configuration (credentials + mode), no code change.
Status lifecycle	queued → mock_sent | sent (accepted by Meta, wamid stored) → delivered → read; or failed (with error_code). Terminal: delivered/read/failed/mock_sent. Status callbacks may arrive out of order — only forward transitions are applied.
Timing	Defaults from PRD §13.7, stored in config.notification_rules (offset_minutes, anchor, sequence_no) and editable per tenant; never in workflow JSON.
Common failure handling	Bounded retry (3 attempts, 30 s / 2 min / 10 min) for 429/5xx/timeouts; permanent errors (invalid number, template rejected/paused, opted-out) are not retried; final failure → status=failed + notify.admin_alerts + visible in delivery log; a failed message never reverses the transaction (BR-NTF-02).
Common audit logging	Every message writes: notify.notification_messages (who/what/why, dedupe_key, correlation_id), notify.whatsapp_messages (template, wamid, phone hash), notify.notification_delivery_events (append-only status history), notify.workflow_executions (n8n run), and audit.audit_logs entry with actor_type=automation. Recipient phone is stored hashed/masked; message body variables retained per retention policy.

9.2 Workflows by domain

Template variable names are the declared logical names in config.notification_templates.variables; the send step maps them to Meta's positional {{1}}…{{n}} in the declared order. Default timing values are development defaults pending client validation (A-08).

Leads

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Lead Created (P0)	lead.created (immediate)	Lead (phone from form; consent given)	`lead_confirmation`	lead_name, studio_name, interest, contact_number	lead-confirmation.json → prepare → send	No consent ⇒ no send (logged). Failure ⇒ alert; lead retained	message + consent ref + lead_id
Follow-Up (P2)	lead.followup_due (schedule, e.g. +1 d, +3 d); stops on lead.converted	Lead (opted-in)	`lead_followup`	lead_name, studio_name, offer_text?, cta_link	Sequence timers; each step re-checks lead status and consent	Lead converted/opted out ⇒ send:false; failure ⇒ skip step, alert	sequence_no in dedupe_key
Trial Reminder (P2)	trial.reminder_due (e.g. 24 h, 2 h before trial class)	Lead / trial registrant	`trial_reminder`	lead_name, class_name, class_time, studio_address	'due' query → send	Trial cancelled ⇒ skip	session_id reference

Memberships

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Purchase Confirmation (P0)	membership.activated (and payment.confirmed when approval already granted)	Member	`membership_activated`	member_name, plan_name, start_date, end_date, studio_name	payment-received.json / activation notice	Approval pending ⇒ 'payment received, awaiting approval' variant (payment_received) instead	membership_id, payment_id
Renewal Reminder (P0)	membership.expiring at 7 d and 3 d and 1 d (schedule)	Member (active membership)	`renewal_reminder`	member_name, plan_name, expiry_date, days_left, renew_link	renewal-reminders.json daily 'due' query	Already renewed ⇒ send:false; failure ⇒ alert (missed renewal risk highlighted)	sequence_no 1..3
Expiry Reminder (P0)	membership.expired / expiry day	Member	`membership_expiry_notice`	member_name, plan_name, expiry_date, renew_link	Expiry-day notice; P2 status check + escalation to admin	As common	membership_id

Bookings

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Booking Confirmation (P0)	booking.confirmed / booking.waitlisted	Member	`booking_confirmation (waitlist variant: booking_waitlisted)`	member_name, class_name, session_datetime, trainer_name, studio_name	booking-confirmation.json	Booking cancelled before send ⇒ skip	booking_id
Cancellation (P1)	booking.cancelled; session.cancelled (studio-initiated)	Member	`booking_cancellation`	member_name, class_name, session_datetime, reason?	Immediate message	As common	booking_id, actor
Class Reminder (P0)	class.reminder at −12 h and −1 h (configurable)	Member with confirmed booking	`class_reminder`	member_name, class_name, session_datetime, location	class-reminders.json schedule + 'due' query	Cancelled ⇒ skip; duplicate ⇒ skip	sequence_no 1..2

Attendance

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Attendance Confirmation (P1 optional)	attendance.recorded (method qr/trainer/front_desk)	Member	`attendance_confirmation`	member_name, class_name, date, feedback_link?	Optional post-class note / feedback request (feature flag)	Non-transactional parts require opt-in	attendance_id
Missed Session Reminder (P2)	attendance.missed (booked, absent) or member.inactive_3d / 7d	Member (opted-in)	`missed_session_nudge`	member_name, class_name, next_session_link	Nudge sequence at 3 d and 7 d	Opt-out ⇒ skip; failure ⇒ log	reason: missed|inactive

Payments

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Payment Success (P0)	payment.confirmed	Payer (member or registrant)	`payment_received`	member_name, amount, currency, purpose, reference	payment-received.json	As common; never delays payment state	payment_id
Payment Failure (P1)	payment.failed	Payer + admin exception task	`payment_failed`	member_name, amount, purpose, retry_link	Message + admin alert	As common; admin task always created	payment_id, failure_code

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Refund Processed (P2)	payment.refunded	Payer	`refund_processed`	member_name, refund_amount, original_reference, eta_days	Message after Razorpay refund.processed	As common	refund_id

Events

Flow	Trigger	Recipient	Template code	Template variables	Workflow	Failure handling	Audit
Registration (P0)	event.registration.confirmed (free: immediate; paid: after verified payment)	Member or external registrant	`event_registration`	name, event_name, event_datetime, venue, studio_name	event-registration.json	Hold expired ⇒ event_hold_expired variant (P1)	registration_id
Reminder (P2)	event.reminder at −24 h, −6 h, −1 h (workshops −24 h/−12 h/−1 h)	Confirmed registrants	`event_reminder`	name, event_name, event_datetime, venue	‘due’ query per sequence	Cancelled ⇒ skip	sequence_no

10. Email & In-App Notifications

Email (transactional first; campaign email only from approved Marketing Agent drafts) and the member in-app centre are **Phase 2** (PRD Parts B/D). Staff/admin in-app alerts (notify.admin_alerts) and the delivery log are Phase 1. Contracts are provider-agnostic (email provider TBD). Delivery rules common to all: templates from config.notification_templates (channel email|in_app); suppression list and bounces honoured (notify.suppression_list); transactional messages ignore marketing opt-out but respect suppression; one message per event/recipient/channel (same dedupe key); failures never reverse transactions.

Notification	Trigger	Recipient	Channel	Workflow	Delivery rules	Pri
Welcome — member	Member account first created / OTP registration completed	Member	WhatsApp (P0 via registration confirmation), email + in-app (P2)	Outbox member.created → send-message (alternate branch)	Once per member; email only if verified address; sent within 5 min	P2
Welcome — staff	Staff invite created (POST /staff)	Staff user	Email (system email sent by API directly — not marketing; required for invite/reset)	API → email adapter with single-use token link (72 h)	Critical path: 3 attempts; on failure Owner sees 'invite not delivered' alert	P0 (invite /reset e-mail)
Membership notifications	membership.activated · .expiring (7/3/1 d) · .expired · pending_approval (staff)	Member; approvers (Owner/Admin) for pending approvals	WhatsApp (P0) · email + in-app (P2)	renewal-reminders.json / send-message	Reminders per notification_rules; quiet hours 21:00–08:00 local for non-critical reminders (config); stop when renewed	P0 Whats App / P2 others
Booking notifications	booking.confirmed · waitlisted · cancelled · waitlist.promoted · class.reminder · session.cancelled	Member; trainer for roster changes (P2)	WhatsApp (P0) · in-app (P2)	booking-confirmation.json / class-reminders.json	Promotion notices are time-critical: send immediately, include claim window; reminders skipped if cancelled	P0 / P2
Payment notifications	payment.confirmed · payment.failed · payment.refunded	Payer; admin for failures	WhatsApp (P0) · email receipt (P2)	payment-received.json	Receipt email attaches invoice link when invoicing exists (P2); failed payments always alert admin in-app	P0 / P2
Event notifications	event.registered · registration.confirmed · reminder · cancelled · hold_expired	Registrant (member or external)	WhatsApp (P0 confirmation) · email + in-app (P2)	event-registration.json	External registrants: consent captured at registration; reminders 24 h/6 h/1 h (P2)	P0 / P2
Staff notifications	Failed message/workflow, template rejected, outbox dead-letter, payment exception, low-attendance alert (Operations Agent P2), pending approvals, reconciliation findings	Owner/Admin (by permission notifications:read); trainers for their session changes	In-app admin alert centre (P0 list via GET /b2b/notifications/alerts) · email digest (P2)	API creates notify.admin_alerts; n8n failure-alert.json	Severity info|warning|critical; critical persists until acknowledged (POST /b2b/notifications/alerts/{id}/ack); dedupe by code+subject within 1 h	P0 in-app / P2 email
AI notifications	Pending AI action awaiting confirmation (in chat), agent decision awaiting approval (P2), weekly feedback theme summary ready, Studio Insights briefing (P3), AI quota reaching 80%	Requesting admin / Owner	In-app (chat card, approval queue) · email (P2)	AI module creates alert/notification rows; no WhatsApp to members without human approval	AI-generated member-facing messages are drafts requiring approval by default; expiry reminders for pending actions at 8 of 10 min	P1 / P2

10.1 Supporting endpoints (email and in-app)

Endpoint	Purpose	Notes	Pri
`POST /internal/notifications/email/send`	Send templated email via provider adapter (called by API/n8n)	Body { notification_id, template_code, to_ref, variables }; provider abstraction; suppression check; result recorded in notify.email_logs	P2 (staff invite/reset e-mail P0)
`POST /api/webhooks/email`	Provider bounce/complaint/delivered events	Signature verified per provider; bounce/complaint adds to suppression list	P2
`GET /b2b/notifications/alerts · POST /b2b/notifications/alerts/{id}/ack`	Admin alert centre	cursor pagination; filter[status], filter[severity]	P0

Endpoint	Purpose	Notes	Pri
`GET /b2c/notifications · POST …/read`	Member in-app centre	See §4.8	P2
`GET/PUT /b2c/profile/communication-preferences`	Opt-in/out per channel/category	Appends notify.communication_consents rows	P1

11. Payment Integration (Razorpay)

Razorpay is the gateway (BR-PAY-01); Phase 1 runs on **test-mode keys** selected by environment variable (production is a configuration change). Only a verified server-side confirmation marks a payment Paid (BR-PAY-02). Payment state and membership approval state stay independent (BR-PAY-03). Amounts in the App API are decimal INR; the Razorpay adapter converts to integer paise. Secrets (key_id/key_secret, webhook secret) live in the secret store per environment.

11.1 Payment state machine

From	Event	To	Rule
—	POST /payments/orders	created	ERP payment record + Razorpay order created; amount server-derived
created	client checkout started / client callback received	pending	Never paid on client evidence
created | pending	webhook payment.captured (verified, amount+currency+order match)	paid	ERP posting via payment.confirm_payment; then outbox payment.confirmed
created | pending	webhook payment.failed	failed	Retry allowed by creating a new order or reusing order until expiry
pending (>15 min)	status-fetch reconciliation	paid | failed | pending	GET /v1/payments/{id} or /v1/orders/{id}/payments; unresolved stays Pending and appears in reconciliation findings
paid	refund.processed	refunded (partial keeps paid with refunded_amount)	P2

11.2 Contracts

Contract	Direction / endpoint	Request	Response / processing	Validation & errors	Pri
Order Creation	API → Razorpay POST https://api.razorpay.com/v1/orders (Basic auth key_id:key_secret). Internal trigger: POST /api/v1/{b2b|b2c}/payments/orders (§3.9, §4.6)	{ amount: <paise>, currency: "INR", receipt: <internal payment id>, notes:{ purpose, tenant_id, member_id|registrant_id, target_id } }	Razorpay { id: order_..., status: created } → stored in payments (provider_ref=order id); API returns { payment_id, razorpay_order_id, amount, currency, key_id } to checkout	amount>0, currency INR; target unpaid; Idempotency-Key ⇒ same order reused; Razorpay 4xx ⇒ 502 PAYMENT_GATEWAY_ERROR; timeout ⇒ 504 with no ERP record left dangling (compensate)	P0
Payment Verification (client hint)	Browser → API POST /b2c/payments/{id}/client-callback	{ razorpay_payment_id, razorpay_order_id, razorpay_signature }	API verifies signature = HMAC-SHA256(order_id + "|" + payment_id, key_secret); on success records 'pending – awaiting webhook' and may fetch payment from Razorpay; response status pending unless authoritative capture confirmed	Invalid ⇒ 422 SIGNATURE_INVALID (audit security event); UI polls GET /payments/{id}	P0
Payment Verification (authoritative)	Razorpay → API POST /api/webhooks/razorpay (§12.1)	Razorpay event payload	Signature verified ⇒ dedupe on event id + razorpay_payment_id ⇒ amount/currency/order cross-check ⇒ ERP payment.confirm_payment ⇒ outbox payment.confirmed	Mismatch ⇒ status pending + finding 'AMOUNT_MISMATCH' + alert; unknown order ⇒ 200 ignore + log	P0
Manual/offline	POST /b2b/payments/manual (§3.9)	—	ERP payment.record_payment; source=manual	actor + reference required	P0
Refunds	API → Razorpay POST /v1/payments/{payment_id}/refund; webhook refund.processed / refund.failed	{ amount (paise), speed: normal, notes:{ reason, refund_id } }	202 processing → webhook updates status; ERP posting; outbox payment.refunded	amount ≤ paid − refunded; permission payments:refund; audited (BR-PAY-06); 422 REFUND_EXCEEDS_PAID; 502	P2
Subscriptions	Two separate uses: (a) member recurring debit — evaluated in Phase 2, opt-in; (b) SaaS tier billing (Basic/Premium/Ultra) — Phase 2	(a) POST /b2c/payments/mandates {plan_id}; (b) POST /platform/tenants/{id}/billing	Razorpay Subscriptions API (plan, subscription, authorization link); webhooks subscription.activated|charged|halted|cancelled mapped to payments and membership renewals	Compliance with authorisation rules (NFR-CMP-03); never mix member plans and platform tiers (BR-MSP-01)	P2

Contract	Direction / endpoint	Request	Response / processing	Validation & errors	Pri
Webhooks	See §12.1	—	—	—	P0

```
// POST /api/v1/b2c/memberships/purchase → 201
{ "success": true, "data": { "membership_id": "MSP-00310", "status": "pending_payment",
  "payment": { "payment_id": "PAY-00877", "razorpay_order_id": "order_Nk3…", "amount": "1500.00", "currency": "INR", "key_id": "rzp_test_…" } } }
// Razorpay Checkout success handler → POST /b2c/payments/PAY-00877/client-callback → 202 { "status": "pending" }
// Later: webhook payment.captured → payment "paid" → membership pending_approval or active (per plan approval rule)
```

12. Webhook Specifications

All inbound webhooks share: HTTPS only; raw body preserved for signature verification; timestamp/replay window (±5 min where a timestamp exists); idempotent processing keyed on provider event ID in ops.webhook_inbox (UNIQUE provider, provider_event_id); the receiver returns 2xx quickly after durable persistence and processes asynchronously where possible; invalid signature ⇒ 401 and discard (security event); unknown tenant ⇒ 200 with no action (avoids retry storms) and log.

12.1 Razorpay (inbound)

Item	Specification
Endpoint	POST /api/webhooks/razorpay (Module Spec §9.3). Tenant resolved from order notes.tenant_id / receipt lookup, then verified against payment record; tenant-specific webhook secrets supported by secret_ref.
Headers	X-Razorpay-Signature (HMAC-SHA256 hex of raw body with webhook secret), X-Razorpay-Event-Id (dedupe key).
Events handled	payment.captured (P0), payment.failed (P0), order.paid (P0, cross-check), refund.processed / refund.failed (P2), subscription.* (P2). Others ⇒ 200 ignored.
Payload (abridged)	{ event: "payment.captured", payload: { payment: { entity: { id, order_id, amount, currency, status, method, notes:{tenant_id,…} } } }, created_at }
Processing	1) verify signature 2) insert webhook_inbox (dedupe) 3) load payment by order_id 4) validate amount/currency/status 5) ERP confirm_payment 6) outbox payment.confirmed 7) mark inbox processed; step 5 failure ⇒ inbox status=failed, retry job 5× (30 s→30 min), payment remains Pending.
Response	200 { received: true } on accepted or duplicate; 401 invalid signature; 500 only if persistence fails (Razorpay will retry).

12.2 ERP events (inbound callbacks)

Item	Specification
Endpoint	POST /internal/events/{type} with type ∈ membership.expiring, membership.expired, session.starting, session.full, event.registration.expired (Module Spec §9.2; last item added by this document).
Security	X-WS-Signature = HMAC-SHA256(internal secret, timestamp + "." + body), X-WS-Timestamp, X-WS-Tenant. Source-IP allow-list of Frappe Cloud egress where available. Internal-only route.
Payload	Envelope of §5.3: { event_id, type, schema_version, tenant_id, occurred_at, subject, data, correlation_id }.
Processing	Validate → audit → write outbox row (same transaction) → 200 { accepted }. Duplicate event_id ⇒ 200 no-op.
Retry	Sender (Frappe) retries 6× exponential up to 24 h; nightly reconciliation covers residual gaps.

12.3 Notification events (WhatsApp status webhook, inbound)

Item	Specification
Endpoints	GET /api/webhooks/whatsapp (Meta verification: hub.mode, hub.verify_token, hub.challenge ⇒ echo challenge) and POST /api/webhooks/whatsapp (status and inbound-message events). Phase 1 mock mode: no Meta calls; endpoint deployed but idle.
Security	X-Hub-Signature-256 = sha256 HMAC of raw body with the Meta app secret (per tenant/app reference); verify token stored in secret store.
Statuses handled	sent, delivered, read, failed (with errors[].code, title). Matched to notify.whatsapp_messages by wamid (UNIQUE provider_message_id) ⇒ append notification_delivery_events ⇒ update notification_messages.status forward-only. Template events (message_template_status_update: approved/rejected/paused) ⇒ update config.notification_templates.approval_status and raise admin alert.
Inbound user messages	Stored minimally for STOP/opt-out keywords (e.g. STOP ⇒ consent withdrawn, notify.communication_consents append); other inbound content not processed in Phase 1.
Response	200 quickly; unmatched wamid ⇒ 200, logged.

12.4 Outbound event webhooks (booking, membership, attendance, notification)

Primary consumer is n8n over the outbox path (§8.1, P0). A tenant-configurable outbound webhook subscription (for a studio's own tools) is **P2**: POST /b2b/webhooks/endpoints { url (https), event_types[], secret } → returns endpoint id; secrets write-only; same envelope and signature scheme as §8.1; delivery attempts recorded and retried like the dispatcher; endpoint auto-disabled after 50 consecutive failures with an admin alert. Contracts (envelope.data):

Family	Event types	data payload	Idempotency / ordering
Booking events	booking.confirmed, booking.waitlisted, booking.cancelled, waitlist.promoted, session.full	{ booking_id, member_id, session_id, starts_at, status, waitlist_position?, cancelled_by?, cancel_reason? }	event_id; booking_id + status version
Membership events	membership.pending_approval, membership.activated, membership.expiring, membership.expired, membership.suspended	{ membership_id, member_id, plan_id, plan_name, status, start_date, end_date, days_left?, payment_status, approval_status }	event_id; membership_id + status version
Attendance events	attendance.recorded, attendance.corrected, attendance.missed	{ attendance_id, session_id, member_id, status, method, corrected_from?, corrected_by? }	event_id; attendance_id
Notification events	notification.sent, notification.delivered, notification.failed, notification.template_rejected	{ notification_id, template_code, channel, status, error_code?, event_ref }	event_id; notification_id + status
Payment / ERP events	payment.confirmed, payment.failed, payment.refunded; erp.sync_failed	{ payment_id, purpose, target_id, amount, currency, status } / { reason, correlation_id }	event_id; payment_id

13. Error Handling & Security

13.1 Error code catalogue

Codes are stable API contract. ‘Retry’ = client may retry (with backoff). Messages shown to users are safe and actionable; details never leak internals or other tenants’ data.

13.2 Validation errors

Code	HTTP	Retry	Meaning / trigger
MALFORMED_REQUEST	400	No	Invalid JSON/encoding, missing body
VALIDATION_FAILED	422	No	One or more fields invalid; details[] has field, code (REQUIRED, INVALID_FORMAT, OUT_OF_RANGE, INVALID_ENUM, TOO_LONG…), message
UNKNOWN_FILTER / UNKNOWN_SORT / FIELD_NOT_EDITABLE	422	No	Unsupported query parameter or attempt to edit protected field
CONFLICT_DUPLICATE	409	No	Duplicate email/phone/lead; details.field and existing_id (id masked for Front Desk if policy)
INVALID_STATE_TRANSITION	409	No	Operation not allowed in current state (e.g. approve an Active membership)
IDEMPOTENCY_KEY_REUSED / PRECONDITION_FAILED	409/412	No	Same key with different body; stale ETag
PAYLOAD_TOO_LARGE / UNSUPPORTED_MEDIA_TYPE	413/415	No	Body or upload limits exceeded

13.3 Authentication errors

Code	HTTP	Retry	Meaning / trigger
AUTH_INVALID_CREDENTIALS	401	No	Wrong e-mail/password/id_token (generic message; no enumeration)
AUTH_TOKEN_EXPIRED / AUTH_TOKEN_INVALID	401	Refresh	Access token expired or malformed; client calls /auth/refresh once
AUTH_REFRESH_INVALID / AUTH_REFRESH_REUSED	401	No	Refresh token unknown/expired; reuse detected ⇒ session family revoked
AUTH_ACCOUNT_LOCKED	423	After lockout	5 failed logins in 15 min; retry_after_seconds provided
AUTH_ACCOUNT_DEACTIVATED	403	No	User deactivated (history retained)
AUTH_MFA_REQUIRED / MFA_INVALID	401	No	Privileged staff MFA challenge (P2 enforcement)
OTP_INVALID / OTP_EXPIRED / OTP_RATE_LIMITED / OTP_PROVIDER_FAILED	401/410/429/502	Varies	OTP verification and delivery problems; provider failure falls back to retry or alternative method

13.4 Authorization errors

Code	HTTP	Retry	Meaning / trigger
PERMISSION_DENIED	403	No	Role/permission/grant lacks the action; details.required_permission (staff only)
NOT_ASSIGNED	403	No	Trainer not assigned to the session (BR-ATT-05)
FEATURE_NOT_ENTITLED / AI_FEATURE_DISABLED	403	No	Tier or feature flag disables the capability (BR-AI-11)
NOT_FOUND	404	No	Resource missing **or belongs to another tenant/member** (deliberately indistinguishable); cross-tenant attempts are audit-logged
TENANT_MISMATCH → returned as NOT_FOUND	404	No	JWT tid ≠ host tenant; security event raised

13.5 Business-rule errors (ERP-originated, mapped 1:1)

Code	HTTP	Retry	Meaning / trigger
BOOKING_CLASS_FULL	409	No	No seat left; waitlist option offered in details.waitlist_available
BOOKING_NOT_ELIGIBLE	422	No	details.reason ∈ no_membership, membership_expired, membership_suspended, plan_excludes_class, limit_reached, approval_pending
BOOKING_DUPLICATE / BOOKING_CUTOFF_PASSED	409	No	Already booked; cancellation inside cutoff window (cutoff_at in details)
ATTENDANCE_DUPLICATE / ATTENDANCE_NOT_ELIGIBLE / SESSION_NOT_OPEN / QR_INVALID / QR_EXPIRED	409/422/410	No	Attendance rules
CAPACITY_BELOW_BOOKINGS / EVENT_FULL / EVENT_REGISTRATION_CLOSED / EVENT_AUDIENCE_RESTRICTED	409/403	No	Capacity and event rules
MEMBERSHIP_OVERLAP / LAST_OWNER / LEAD_ALREADY_CONVERTED / TRAINER_CONFLICT	409	No	Domain conflicts

13.6 ERP errors

Code	HTTP	Retry	Meaning / trigger
ERP_UNAVAILABLE	503	Yes	Connection refused/circuit open; UI shows clear failure; nothing cached presented as confirmed (NFR-AVL-03)
ERP_TIMEOUT	504	Query first	Write timed out — outcome unknown; details.outcome=unknown, idempotency key provided; client must re-read state before retrying write
ERP_ERROR	502	Maybe	Unexpected ERP failure (traceback logged, not returned)
ERP_CONTRACT_MISMATCH	503	No	Adapter ↔ custom-app version incompatibility detected; alerts on-call
ERP_PROVISIONING_FAILED	502	Yes (idempotent)	Tenant provisioning saga failed and was compensated

13.7 AI, payment and notification errors

Code	HTTP	Retry	Meaning / trigger
AI_UNAVAILABLE / AI_TIMEOUT	503/504	Yes	LLM provider down/slow after one bounded retry; dashboards unaffected
AI_RATE_LIMITED / AI_QUOTA_EXCEEDED	429	After reset	Per-user rate or per-tenant token/cost quota
AI_SESSION_NOT_FOUND / AI_ACTION_EXPIRED / AI_ACTION_ALREADY_DECIDED	404/410/409	No	Session or pending action lifecycle
AI_TOOL_DENIED / AGENT_TOOL_DENIED	403	No	Tool not permitted for principal at execution time
PAYMENT_GATEWAY_ERROR	502	Yes	Razorpay order/refund call failed; no payment marked Paid
SIGNATURE_INVALID / WEBHOOK_REPLAY	401	No	Webhook/callback signature or timestamp check failed
ALREADY_PAID / REFUND_EXCEEDS_PAID / AMOUNT_MISMATCH	409/422	No	Payment integrity checks (AMOUNT_MISMATCH raised internally as finding)
NOTIFICATION_NOT_SENT (delivery record codes)	n/a	Bounded	Recorded, not returned to transactions: OPTED_OUT, NO_PHONE, TEMPLATE_NOT_APPROVED, TEMPLATE_REJECTED, WHATSAPP_RATE_LIMITED, RECIPIENT_UNREACHABLE, DUPLICATE_SEND, PROVIDER_ERROR
RETRY_NOT_ALLOWED	409	No	Message not in failed state or attempts exhausted

13.8 Security controls

Control	Specification
Authentication	Staff: e-mail + password (Argon2id, mandatory first-login reset, lockout, MFA P2 before production). Members: phone OTP (6 digits, 5 min, 5 attempts, hashed) + optional Google. Visitors: none. Services: signed credentials, rotated, scoped. Sessions: 15-min access JWT + rotating refresh (§2.1).
Authorization	Central policy service evaluates identity → role → permission → tenant/studio/branch → ownership on every request; UI hiding is cosmetic; AI/agent tools re-checked at execution; non-bypassable ERP rules.
Tenant isolation	Host-resolved tenant, JWT tid, request-scoped context, PostgreSQL RLS (SET LOCAL app.tenant_id), adapter-injected ERP tenant with permission_query_conditions, tenant-filtered AI retrieval, per-tenant n8n credentials/templates; automated cross-tenant tests in CI on every module; denials ⇒ 404 + audit + alert on repetition.
Audit logging	Append-only audit.audit_logs (tenant, actor, actor_type human|automation|ai, action, resource, before/after, timestamp, request context, correlation_id) written in the same transaction as the outbox event; covers approvals, payment changes, attendance corrections, role/permission/config changes, deletions, tenant-boundary denials, every AI tool call, every automated send. Application role has no UPDATE/DELETE grant.
Encryption	TLS 1.2+ everywhere with HSTS; at-rest encryption for PostgreSQL volumes/backups and object storage; envelope encryption for phone numbers/OTP secrets/webhook secrets where stored; passwords hashed; refresh tokens and OTPs stored hashed; secrets only in secret store (write-only in UI, never in Git or client); PII masked in logs and prompts.
Input & transport	Schema validation on all inputs (DTOs generated from OpenAPI), strict CORS allow-list, security headers (CSP, X-Content-Type-Options, frame-ancestors), request-size limits, upload scanning, output encoding, parameterised queries only.
Webhook security	Signature + timestamp + idempotency on every webhook (Razorpay HMAC-SHA256, Meta X-Hub-Signature-256, internal HMAC); replay window ±5 min; secrets per environment/tenant.
Payment security	Status changes only from server-verified confirmations; amounts server-derived; no card data touches the platform (Razorpay Checkout).
Data protection	Consent capture, purpose limitation, retention and deletion per audit.data_retention_policies and data-subject-request handling (P2); legal review pending (NFR-CMP-01).

Rate limits (defaults; per-tenant tunable)

Scope	Limit	On breach
Global per IP (unauthenticated)	120 req/min	429 RATE_LIMITED + Retry-After
Authenticated user (staff)	600 req/min; writes 120/min	429
Member	300 req/min; bookings 20/min; payments 10/min	429
Login / password reset	10/min/IP; 5 failures/15 min/account; 3 resets/hour/e-mail	429/423
OTP request / verify	5/hour/phone; 10/hour/IP; 5 verify attempts/challenge	429 OTP_RATE_LIMITED
Public forms & enquiry chat	5 submissions/hour/IP; chat per §6.3; captcha required	429/403
AI endpoints	Per §6 tables + per-tenant daily token quota by tier	429 AI_QUOTA_EXCEEDED
Reports / exports	30/min; exports 5/hour/user	429
Webhooks (inbound)	Not throttled per provider; abusive volume from non-allow-listed IPs limited at proxy	—
Internal automation API	600 req/min per service principal	429 (n8n retries)

14. MVP API Scope

P0 = mandatory for the 5-Day MVP · P1 = important (first sprint after MVP / Phase 1 additions) · P2 = future (Phase 2 and later). Classification follows PRD §16.1 (must-complete list and cut order), Module Spec §10 and Document 4. If time slips, AI is cut first in the PRD's order: booking suggestions/recommendations → feedback → enquiry → paid-event payment → Create Member → read queries; core modules, permissions, tenant isolation and audit are never cut.

14.1 Core APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Auth & RBAC	staff login, member OTP request/verify, staff invite accept, refresh, logout, password forgot/reset/change, /auth/me, permission catalogue, role assignment, grants CRUD	Google sign-in/link, custom roles, session revoke	MFA enroll/verify
Tenant	POST/GET /platform/tenants, feature-flag override, /public/tenant	tier change, suspend, studios/branches CRUD	self-service onboarding & billing
Dashboard & Reports	kpis, quick-actions, today, core reports (6), report catalogue	kpi-snapshots	pipeline drill-down, exports
CRM & Leads	leads list/create/detail/update/convert, /public/enquiries	merge	source attribution reports, trial flow
Members	list/create/detail/update/archive	history tabs	bulk import
Memberships	plans CRUD, assign, approve, renew, expiring list; B2C plans/purchase/history	suspend/cancel	freeze/unfreeze, upgrade
Classes	class types, schedules, sessions, roster, QR	cancel session	blackout, substitution, rooms
Bookings	book (staff & member), cancel, list, eligibility, leave waitlist	manual promote	conversational booking
Attendance	check-in (QR/front desk), trainer mark, correct, list, member history	session summary	attendance summary/streaks
Events	event CRUD, registrations (staff, member, public), list	cancel registration	waitlist, reminders config
Payments	orders, client-callback, status poll, manual record, list/detail (incl. Razorpay test mode)	verify-status, reconciliation list	refunds, receipts/invoices, mandates
Staff & Settings	staff CRUD/deactivate/assign, settings, branding, event categories, feature-flags (read)	FAQ, studios/branches, audit view	integrations screen
Profile	get/patch profile, phone change	communication preferences, Google link	—

14.2 ERP APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Adapter → custom app	create_member, create_lead/convert_lead, create_booking/cancel_booking/promote_waitlist, check_in/mark_attendance, activate/expire/suspend_membership, event.register, record_payment/confirm_payment, tenant.provision, system.contract_version	report.<code> aggregates, staff records	invoicing/GST posting, reconciliation helpers
ERP → API callbacks	membership.expiring/expired, session.full (signed)	session.starting, event.registration.expired	—
Reconciliation	—	nightly job E13	advanced reconciliation

14.3 AI APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Admin Assistant	sessions, messages (2 read queries + Create Member), confirm/reject, /ai/audit	session get/delete	more actions with approval queue, insights
Enquiry	sessions, messages, lead capture (cut order 3)	handoff	multi-channel
Booking Assistant	suggest (cut order 1)	chat form	propose/confirm booking

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Recommendation	POST /b2c/ai/recommendations, GET /b2c/recommendations (cut order 1)	feedback signal	AI-personalised
Feedback	—	submit, list, themes, classify job (cut order 2)	reclassify, trends

14.4 Agent APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Common runtime	— (architecture only: tables, principals, tool registry)	service principals, agent tool grants	trigger/execution/status/result/decision APIs
CRM, Operations	—	—	P2 phase delivery
Marketing, Advertising, Growth, Insights	—	—	P3 phase delivery
Session Summary	—	—	Future

14.5 Automation APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
Outbox → n8n	envelope, dispatcher, retry, dead-letter	replay endpoint	outbound tenant webhooks
n8n → API	messages/prepare, messages/result, executions, alerts, /due (class & renewal), events intake	automation/events	—
Workflows	lead confirmation, booking confirmation, class reminders, renewal reminders, payment received, event registration confirmation, send-message (mock), failure-alert	attendance feedback link	lead follow-up, trial reminder, event reminders, inactivity nudges, waitlist promotion message

14.6 Communication APIs

Group	P0 — 5-Day MVP	P1 — Important	P2 — Future
WhatsApp	mock mode, delivery log, templates & rules config, Lead Created, Purchase Confirmation, Renewal/Expiry Reminder, Booking Confirmation, Class Reminder, Payment Success, Event Registration	Cancellation, Payment Failure, Attendance confirmation, retry endpoint, WhatsApp status webhook (live), live credentials swap	Follow-up, Trial Reminder, Missed Session, Refund Processed, Event Reminder
Email	Staff invite / password-reset e-mail (system e-mail only)	—	Welcome, receipts, renewals, digests, bounce webhook
In-app	Admin alert list/ack	WhatsApp history (member)	Member notification centre, preferences UI
Payments gateway	Razorpay order creation, verification, webhook (test mode)	status-fetch reconciliation	refunds, subscriptions, production keys

14.7 Delivery mapping (PRD 5-day plan) and ownership

Day	API deliverables (P0)	Owner (Architecture §14.7 proposal)
Day 1	Envelope/error standards, OpenAPI skeleton, auth, tenant guard, RBAC, audit, outbox, ERP adapter + contract_version, tenant provisioning	Tahir (API foundations, ERP adapter); Member 5 (custom app)
Day 2	Members, membership plans/assign/approve/renew, settings/branding, staff; AI gateway + Assistant skeleton	Harish (domain modules); Member 6 (portals); Shreya/Harshini (AI)
Day 3	Class types/sessions, bookings, attendance, roster/QR; AI read tools	Harish; Member 5 (atomic ERP methods); Harshini

Day	API deliverables (P0)	Owner (Architecture §14.7 proposal)
Day 4	Payments (Razorpay test) + webhook, CRM/leads, dashboard KPIs, B2C endpoints, n8n prepare/result/due + mock workflows; Create Member	Member 6 (checkout/payments UI); Harish (n8n); Tahir (webhooks); Harshini
Day 5	Events, public enquiry & Enquiry/Feedback/Recommendation/Booking-suggestion assistants, E2E canonical flow, cross-tenant and contract tests	All; QA coordinated by Tahir

14.8 QA and integration acceptance checklist

- OpenAPI validity and contract tests (schemathesis-style) pass for every P0 endpoint; generated client compiles with zero type errors.
- Envelope, error codes, pagination, idempotency and correlation-ID propagation verified (UI → API → ERP → outbox → n8n → WhatsApp mock).
- Cross-tenant suite: every module returns 404 and logs a security event for foreign IDs (host, JWT, body, AI tool, n8n paths).
- 50 parallel last-seat booking requests → exactly one confirmation (S3); booking p95 < 1 s (S4).
- Duplicate Razorpay webhook and duplicate outbox delivery each produce one payment transition and one message (S5); delayed webhook leaves payment Pending.
- ERP down: writes return 503/504 with no fabricated success and no outbox event; recovery verified by reconciliation.
- AI: Create Member scenarios — missing fields, duplicate, permission denied, API failure, expiry — report the true result with zero false success (S6); assistant figures equal dashboard figures.
- WhatsApp mock: each P0 flow writes a would-send record; switching to live requires configuration only.